REAL-TIME CROWD AND VIOLENCE DETECTION USING YOLOV8 ALGORITHM

A PROJECT REPORT

Submitted by

LATHIKESHWARAN S [211421104138]

LOGESH S [211421104145]

NAVEEN S [211421104172]

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2025

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report **"REAL-TIME CROWD AND VIOLENCE DETECTION USING YOLOV8 ALGORITHM"** is the bonafide work of "**LOGESH S, LATHIKESHWARAN S, NAVEEN S"** who carried out the project work under my supervision.

Signature of the HOD with date

DR L.JABASHEELA M.E., Ph.D.,
HEAD OF THE DEPARTMENT

PROFESSOR,

Department of Computer Science
and Engineering,
Panimalar Engineering College,
Chennai - 123

Signature of the Supervisor with date

DR M. KRISHNAMOORTHY M.E., Ph.D.,
SUPERVISOR

PROFESSOR,

Department of Computer Science
and Engineering,
Panimalar Engineering College,
Chennai - 123

Certified that the above candidate(s) was examined in the End Semester Project Viva- Voce Examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION BY THE STUDENT

We LOGESH S, LATHIKESHWARAN S, NAVEEN S hereby declare that this project report titled “Real-time crowd and violence detection using yolov8 algorithm”, under the guidance of Dr. M. KRISHNAMOORTHY M.E., Ph.D., is the original work done by us and we have not plagiarized or submitted to any other degree in any university by us.

NAME OF THE STUDENT(S)

LATHIKESHWARAN S
LOGESH S
NAVEENS

ACKNOWLEDGEMENT

Our profound gratitude is directed towards our esteemed Secretary and Correspondent, **Dr. P. CHINNADURAI, M.A., Ph.D**., for his fervent encouragement. His inspirational support proved instrumental in galvanizing our efforts, ultimately contributing significantly to the successful completion of this project.

We want to express our deep gratitude to our Directors, **Tmt. C. VIJAYARAJESWARI, Dr. C. SAKTHI KUMAR, M.E., Ph.D., and Dr. SARANYASREE SAKTHI KUMAR, B.E., M.B.A., Ph.D.,** for graciously affording us the essential resources and facilities for undertaking of this project.

Our gratitude is also extended to our Principal, **Dr. K. MANI, M.E., Ph.D.,** whose facilitation proved pivotal in the successful completion of this project.

We express our heartfelt thanks to **Dr. L. JABASHEELA, M.E., Ph.D.,** Head of the Department of Computer Science and Engineering, for granting the necessary facilities that contributed to the timely and successful completion of project.

We would like to express our sincere thanks to **Dr. M.. KRISHNAMOORTHY, M.E., Ph.D.,** and **Dr. R. JOSPHINELEELA, M.E., Ph.D.,** all the faculty members of the Department of CSE for their unwavering support for the successful completion of the project.

NAME OF THE STUDENT(S)

LATHIKESHWARAN S
LOGESH S
NAVEENS

ABSTRACT

This project aims to develop a real-time crowd and violence detection system using the state-of-the-art YOLOv8 algorithm. The system monitors live video feeds to detect sudden increases in crowd density and violent activities, providing real-time alerts for quick intervention. By leveraging the YOLOv8 model's advanced object detection capabilities, the system ensures high accuracy and speed in identifying potential security threats in public areas such as gatherings, events, and surveillance zones. This technology improves public safety by allowing authorities to respond quickly and proactively to incidents. The proposed system works by first collecting and preprocessing data from various sources, followed by training the YOLOv8 model on a curated dataset of diverse crowd and violence scenarios. This model is then integrated with a user-friendly interface that allows security personnel to interact with the system, receive notifications, and take action. Upon detecting an anomaly, the system triggers both an alert sound and an email notification, ensuring immediate attention. The system’s ability to track crowd behavior and identify violent actions provides an enhanced security monitoring solution that is scalable and reliable. By combining real-time object detection, crowd density analysis, and violence detection into one integrated system, this project contributes significantly to the field of public safety and surveillance. The system's continuous adaptation through new data and feedback improves its accuracy over time, ensuring it remains effective in evolving environments. Future work will focus on refining detection capabilities, addressing challenges such as false positives in dense environments, and incorporating additional data sources for a more comprehensive monitoring system. This approach ensures that the system can handle diverse public safety challenges in various real-world settings.

LIST OF FIGURES

LIST OF ABBRIVIATION

S.NO.	ABBREVIATION	DEFINITION
1.	AI	Artificial Intelligence
2.	CTTV	Closed-Circuit Television
3.	CNN	Convolutional Neural Network
4.	FPS	Frames Per Second
5.	GPU	Graphics Processing Unit
6.	SMTP	Simple Mail Transfer Protocol
7.	YOLO	You Only Look Once
8.	RNN	Recurrent Neural Network
9.	LSTM	Long Short-Term Memory
10.	API	Application Programming Interface
11.	DFD	Data Flow Diagram
12.	UML	Unified Modeling Language

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 PROBLEM DEFINITION

Surveillance plays a vital role in ensuring public safety, yet existing systems have several shortcomings. One of the major problems with traditional surveillance is the reliance on human operators, who may overlook critical events due to fatigue or distractions. Additionally, manual monitoring is not scalable for large public spaces. The lack of an efficient, automated system for detecting violent activities and sudden crowd movements presents a challenge in modern security frameworks. Even when AI-based solutions are integrated, many existing systems have high false positive rates, leading to unnecessary alerts that reduce overall reliability. Another issue with conventional security systems is their passive nature. CCTV footage is typically reviewed after an incident has occurred, rather than providing real-time intervention. This delay in response can result in loss of life, injuries, and increased crime rates in public spaces. Furthermore, many surveillance solutions are not optimized for real-time performance. They struggle to process high-resolution video feeds efficiently, leading to bottlenecks in data analysis. High processing times mean that potential threats may not be identified in time to prevent escalation. This project seeks to address these challenges by developing a real-time, AI-powered crowd and violence detection system using YOLOv8. The system ensures high-speed analysis, low false positives, and immediate alert mechanisms to improve public safety outcomes.

1.2 REAL-TIME DETECTION AND INFERENCE

The trained YOLOv8 model is deployed into a real-time processing system that continuously analyzes live video feeds to detect and infer aggressive behavior. By integrating with OpenCV, the model captures and processes frames in real-time, ensuring efficient live monitoring. The system achieves a processing rate of 30-45 frames per second (FPS), maintaining smooth and uninterrupted analysis.

The violence detection logic is designed to recognize aggressive hand movements, specific body postures, and sudden crowd surges, enabling the identification of potentially violent activities. To minimize false positives, the system applies a confidence threshold of 0.85, ensuring that only high-probability detections trigger alerts.

To enhance reliability, the system can integrate with alarm mechanisms, automated emergency response systems, or law enforcement notification protocols, ensuring swift action in case of detected threats. Moreover, the architecture can be fine-tuned with adaptive learning techniques, allowing continuous model improvement based on real-world data, thereby increasing detection accuracy over time.

The model can also be integrated with facial recognition systems to identify repeat offenders or individuals flagged for suspicious behavior. To further enhance precision, the system incorporates temporal analysis, examining sequences of frames to differentiate between actual violence and harmless gestures.

1.3 ALERT MECHANISM

The alert mechanism is a crucial component of the real-time violence detection system, ensuring timely responses to potential threats. The system integrates an automatic audio alert feature that plays a pre-configured warning message whenever violent behavior is detected. This immediate response can help deter potential aggressors and alert nearby individuals to the situation. Additionally, email notifications are sent to security personnel, providing real-time updates on detected threats, including location details, timestamps, and confidence scores. These notifications can also include snapshots of the detected event, allowing security teams to assess the severity of the situation before taking action.

The audio alert system can be programmed to escalate based on the severity of the detected aggression, ensuring an appropriate level of response. For instance, minor altercations may trigger a warning message, while highly aggressive behavior may activate emergency sirens or direct police intervention.

The email notification system can be customized to include detailed incident reports, including AI-generated descriptions, affected areas, and recommended actions. Alerts can also be stored in a centralized database, enabling security teams to analyze historical patterns and improve future preventive measures.

Overall, the integration of an automatic audio alert system and email notifications significantly enhances the effectiveness of the violence detection system, providing real-time threat intelligence and facilitating rapid response to potential dangers.

1.4 MODEL OPTIMIZATION

Model optimization is a critical step in enhancing the accuracy and efficiency of the YOLOv8-based violence detection system. Training the model with a diverse dataset containing images and videos from various crowd and violence scenarios ensures better generalization and robustness in real-world applications.

The dataset includes different lighting conditions, camera angles, crowd densities, and variations in aggression levels to improve detection reliability across multiple environments. By incorporating footage from security cameras, public surveillance, and simulated scenarios, the model learns to recognize subtle behavioral cues that indicate potential violence.

To further enhance accuracy, data augmentation techniques such as rotation, scaling, noise addition, and contrast adjustments are applied, ensuring the model performs well in varying conditions. The training process involves fine-tuning YOLOv8's hyperparameters, such as learning rate, anchor box sizes, and confidence thresholds, to optimize detection speed and precision.

Transfer learning techniques allow the model to leverage pre-trained weights from large-scale object detection datasets, accelerating training while maintaining high performance. Additionally, balancing the dataset with equal instances of violent and non-violent scenarios helps prevent bias and reduces false positives.

1.5 USER-FRIENDLY DASHBOARD

A user-friendly dashboard is essential for efficiently monitoring and managing the real-time violence detection system. The interactive web interface provides security personnel with a centralized platform to view live video feeds, analyze detected threats, and manage alerts seamlessly. It features an intuitive design with real-time notifications, allowing users to respond promptly to potential incidents. The dashboard includes adjustable sensitivity settings, enabling operators to fine-tune the detection thresholds based on environmental factors and security requirements.

The interface supports role-based access control, ensuring that only authorized personnel can modify system parameters or access critical data. Integration with mobile-friendly designs allows security teams to monitor alerts remotely via smartphones or tablets.

The dashboard also features interactive graphs and heatmaps, helping users identify high-risk areas based on past incidents. A built-in search and filtering system allows security teams to quickly retrieve specific alerts or analyze patterns over time. Multi-camera support enables simultaneous monitoring of different locations, improving situational awareness.

Customizable alert preferences let users set different notification modes, such as sound alerts, email reports, or SMS updates. Future enhancements may include integration with facial recognition and automated incident reporting to further streamline security operations.

1.6 PURPOSE OF THE PROJECT

The purpose of this project is to develop a real-time, AI-powered crowd and violence detection system using YOLOv8 to enhance public safety and security. The system aims to:

- Automate Threat Detection – Reduce reliance on human operators by using AI to detect violent activities and abnormal crowd movements in real-time.
- Minimize False Positives – Improve the accuracy of detection algorithms to reduce unnecessary alerts and enhance system reliability.
- Enable Immediate Response – Provide instant alerts to security personnel to enable faster intervention and prevent escalation of incidents.
- Enhance Scalability – Support large-scale surveillance in public spaces with optimized processing for high-resolution video feeds.
- Improve Public Safety – Reduce crime rates, injuries, and loss of life by enabling proactive security measures through AI-driven analysis.

By integrating YOLOv8, the system ensures high-speed object detection, real-time performance, and efficient processing of surveillance footage, making it a robust solution for modern security challenges.

1.7 MOTIVATION

Ensuring public safety in crowded areas has become increasingly challenging due to the limitations of traditional surveillance systems. Manual monitoring by human operators is prone to fatigue, distractions, and inefficiencies, often leading to critical events being overlooked. Additionally, most conventional security systems function reactively, analyzing recorded footage only after an incident has occurred, rather than providing real-time intervention. This delay in response can result in increased crime rates, injuries, and loss of life, highlighting the need for a more proactive approach to security.

While AI-based surveillance solutions exist, they often suffer from high false positive rates, leading to unnecessary alerts that reduce their reliability. Moreover, the computational inefficiencies of many existing models make them unsuitable for processing high-resolution video feeds in real-time, limiting their scalability in large public spaces such as airports, railway stations, stadiums, and shopping malls. With the rise of smart cities and AI-driven security frameworks, it is essential to develop an efficient system that not only detects threats accurately but also operates with minimal delay.

Given these shortcomings, there is a strong need for an AI-powered, real-time crowd and violence detection system that can automatically identify threats with high accuracy, minimal false alarms, and optimized computational performance. By leveraging YOLOv8, a state-of-the-art object detection model, this project aims to enhance surveillance capabilities by enabling instant detection, rapid response, and improved security outcomes

CHAPTER 2

LITERATURE SURVEY

[1] **Inbavalli A., Jarshini T., and Muralikrishnaa M.** proposed a concept in which an aggressive behavior detection and alert system was created using deep learning techniques. The system processes video feeds in real time to identify instances of aggressive behavior and trigger alerts. It employs Convolutional Neural Networks (CNNs) and other deep learning algorithms to ensure high accuracy and efficiency. This system enables immediate responses to violence or aggression in public spaces.

"EFFICIENT AGGRESSIVE BEHAVIOUR DETECTION AND ALERT SYSTEM EMPLOYING DEEP LEARNING TECHNIQUES – 2024"

Advantages - The system provides quick response times and high accuracy, helping to prevent violence escalation. By detecting aggressive behavior in real time, it enhances security and safety in public areas.

Disadvantages - The system's performance depends on the quality of the video feed. It may face challenges in highly crowded environments, where detecting individual aggressive actions becomes more difficult.

[2] **Sudharson D., Srinithi J., Akshara S., Abhirami K., Sriharshitha P., and Priyanka K.** proposed a concept in which a proactive headcount and suspicious activity detection system was developed using the YOLOv8 algorithm. The system processes live video feeds to estimate crowd sizes and detect unusual behavior in public spaces. By identifying potential security threats in real time, it enables proactive safety measures.

"PROACTIVE HEADCOUNT AND SUSPICIOUS ACTIVITY DETECTION USING YOLOv8 – 2023"

Advantages - The system is highly effective for crowd control and monitoring. It can identify a variety of suspicious activities with high precision, improving public safety.

Disadvantages - The system may face difficulties in highly congested areas, where distinguishing between normal and suspicious behavior becomes challenging.

[3] **Nasir R., Jalil Z., Nasir M., Noor U., Ashraf M., and Saleem S.** proposed a concept in which an enhanced framework for real-time dense crowd abnormal behavior detection was developed using the YOLOv8 algorithm. The system is designed for dense crowd surveillance, identifying abnormal behaviors such as fights or panic in highly packed areas and triggering alerts for timely intervention. It employs data augmentation and advanced tracking techniques to enhance detection accuracy.

"AN ENHANCED FRAMEWORK FOR REAL-TIME DENSE CROWD ABNORMAL BEHAVIOR DETECTION USING YOLOv8 – 2024"

Advantages - The system is highly effective in crowded environments and significantly reduces false positives, making it reliable for dense crowd monitoring.

Disadvantages - The framework is computationally intensive, requiring powerful hardware for real-time processing and analysis.

[4] **Jyothsna V., Alle C., Kurnutala R., Ganesh K. N., KushalKarthik K. R., and Pydala B.** proposed a concept in which a YOLOv8-based security system was developed to detect individuals, monitor distances, and identify weapons or violent actions in real time. The system enhances security by providing speech alerts and email

notifications to security personnel. By integrating multi-modal data, it improves detection accuracy and response efficiency.

“YOLOv8-BASED PERSON DETECTION, DISTANCE MONITORING, SPEECH ALERTS, AND WEAPON IDENTIFICATION WITH EMAIL NOTIFICATIONS – 2024”

Advantages - The system offers a robust, multi-layered security solution with real-time alerts and notifications, improving situational awareness and response time.

Disadvantages - The complexity of system integration requires significant resources for implementation and ongoing maintenance.

[5] **P., Patil S., and Pattanshetti T.** proposed a concept in which a real-time violence and weapon detection system was developed using advanced deep learning techniques. The system analyzes video footage from CCTV cameras to identify violent actions or weapons and triggers immediate alerts to security personnel. This approach enhances public safety by ensuring rapid response to potential threats.

“REAL-TIME VIOLENCE AND WEAPON DETECTION AND ALERT SYSTEM – 2024”

Advantages - The system achieves high accuracy in detecting threats and significantly reduces response time to incidents, improving public security.

Disadvantages - It may produce false positives in densely populated areas and could miss threats in low-quality video footage.

[6] **Kumar R., Rajpurohit D. S., Hanief M., Sharma S., Choudhury T., and Sar A.** proposed a concept in which a system was developed to detect criminal activities by analyzing live CCTV footage. The system leverages machine learning algorithms to

identify specific criminal activities such as theft, assault, or vandalism and instantly notifies authorities for prompt intervention.

"DETECTION OF CRIMINAL ACTIVITIES/CRIMINAL THROUGH CCTV BY LIVE FOOTAGE ANALYSIS – 2024"

Advantages - The system enables early identification of criminal activities, significantly reducing response time and improving security measures.

Disadvantages - Its accuracy depends on the quality of CCTV footage and environmental conditions, which may affect detection efficiency.

[7] **Bensakhria A.** proposed a concept in which real-time edge AI video analytics was utilized for threat detection in sensitive environments such as airports, government buildings, and military sites. By processing video data at the edge, the system minimizes latency and enhances real-time decision-making, ensuring quicker responses to potential threats.

"LEVERAGING REAL-TIME EDGE AI-VIDEO ANALYTICS TO DETECT AND PREVENT THREATS IN SENSITIVE ENVIRONMENTS – 2023"

Advantages - The system enables real-time processing at the edge, reducing response time and improving overall security efficiency.

Disadvantages - The implementation requires high computational resources and comes with significant costs for deploying and maintaining edge devices.

[8] **Benoit P., Bresson M., Xing Y., Guo W., and Tsourdos A.** proposed a concept in which a real-time vision-based system was developed to detect violent actions through CCTV cameras using pose estimation. By analyzing human body movements, the system identifies aggression or violent behaviors in public spaces, enhancing public safety through automated surveillance.

"REAL-TIME VISION-BASED VIOLENT ACTIONS DETECTION THROUGH CCTV CAMERAS WITH POSE ESTIMATION – 2023"

Advantages - The system provides high precision in detecting violent actions by analyzing body pose and movement patterns.

Disadvantages - It may struggle with detecting actions when bodies are occluded or in low-light conditions, affecting accuracy.

[9] **Reddy K. and Reddy S.** proposed a concept in which a smart monitoring system, **OccupEye**, was developed to track building traffic and animal movement using AI-based video analysis. The system identifies patterns in both human and animal movements, making it useful for wildlife monitoring and efficient building occupancy management.

"OCCUPEYE – BUILDING TRAFFIC AND ANIMAL MONITORING SYSTEM – 2024"

Advantages - The system is versatile, serving dual purposes in wildlife tracking and building management, enhancing efficiency in both domains.

Disadvantages - Implementing the system in complex environments with varied operational conditions may pose challenges to accuracy and reliability.

[10] **Sapkota R., Qureshi R., Calero M. F., Badjugar C., Nepal U., Poulose A., and Karkee M.** proposed a comprehensive review detailing the evolution of the YOLO object detection algorithm from its inception to the latest **YOLOv10** version. The study highlights advancements in accuracy, speed, and diverse applications in real-time object detection tasks, providing a thorough analysis of the algorithm's development.

"YOLOv10 TO ITS GENESIS: A DECADAL AND COMPREHENSIVE REVIEW OF THE YOU ONLY LOOK ONCE (YOLO) SERIES – 2024"

Advantages - Offers an in-depth understanding of the YOLO algorithm, its improvements, and its capabilities across different versions.

Disadvantages - The review focuses on theoretical advancements and is not directly applicable to specific real-world problems without further adaptation.

[11] **Qaraqe M., Yang Y. D., Varghese E. B., Basaran E., and Elzein A.** proposed a concept in which the Swin Transformer was leveraged to analyze crowd size and detect violent behaviors in real-time. The system processes video data to assess both the density and potential threat level of a crowd, improving situational awareness and security management.

“CROWD BEHAVIOR DETECTION: LEVERAGING VIDEO SWIN TRANSFORMER FOR CROWD SIZE AND VIOLENCE LEVEL ANALYSIS – 2024”

Advantages - The use of the Swin Transformer enhances performance in both crowd detection and violence level analysis, ensuring higher accuracy in real-world scenarios.

Disadvantages - The system demands significant computational resources for real-time processing, making deployment challenging in resource-constrained environments.

[12] **Gkountakos K., Ioannidis K., Tsikrika T., Vrochidis S., and Kompatsiaris I.** proposed a system for detecting violence in crowded environments by analyzing video footage with AI-based techniques. The system identifies violent behaviors and alerts authorities, enabling rapid intervention in public spaces.

“CROWD VIOLENCE DETECTION FROM VIDEO FOOTAGE – 2021”

Advantages - Offers a scalable solution for monitoring and ensuring security in densely populated areas.

Disadvantages - May face challenges in high-density crowds where overlapping movements can reduce detection accuracy.

[13] **Chang K.-C. and Liao Y.-C.** proposed a system that utilizes human body pose recognition to detect violent events through CCTV cameras. The system analyzes human movements to identify aggressive behaviors, such as fights, improving the accuracy of violence detection in surveillance environments.

“DESIGN OF VIOLENCE EVENT DETECTION SYSTEM BASED ON CCTVS BY HUMAN BODY POSE RECOGNITION – 2023”

Advantages - Provides more precise detection of violent events by analyzing human body poses, reducing false positives.

Disadvantages - May struggle to detect violent actions in large crowds or when individuals are partially obstructed.

[14] **Hada A. S., Bainsla B. Jatin Singh, Tapaswi S., and Jana B.** proposed an embedded system for real-time societal violence detection and intervention using video analysis. The system integrates various sensors and machine learning models to identify and report violent incidents efficiently.

“EMBEDDED SYSTEM FOR SOCIETAL VIOLENCE DETECTION AND INTERVENTION – 2024”

Advantages - A cost-effective and efficient solution for real-time violence detection, making it suitable for widespread deployment.

Disadvantages - The system’s performance may be constrained by **processing power** and **sensor accuracy**, particularly in complex environments.

[15] **Salau A. O., Daniel N. I., Ayobamidele O., Vohra S. K., Alemran A., and Onibonoje M. O.** proposed an AI-powered security system for hostels that integrates deep learning models to detect smoking and violent behavior**.** The system leverages real-time video analysis to enhance hostel security through automated detection and alerts.

“ENHANCING HOSTEL SECURITY: INTEGRATION OF DEEP LEARNING MODELS FOR SMOKING AND VIOLENCE DETECTION – 2024”

Advantages - A multi-functional system that improves security by detecting both smoking and violent activities in real time.

Disadvantages - May produce false positives and struggles with detection in poorly lit environments**.**

CHAPTER 3

THEORITCAL BACKGROUND

3.1 IMPLEMENTATION ENVIRONMNET

The implementation environment for real-time crowd and violence detection software includes both mobile and computer devices. The hardware requirements include an Intel Core i7 or AMD Ryzen processor or higher, with a minimum of 16GB RAM for both mobile and computer. The storage requirements are at least 512GB SSD for computers and 64GB for mobile devices. Additionally, a camera with 1080p Full HD resolution and a minimum of 30FPS is required. The software requirements include Android OS version 10 for mobile and Windows 10 or higher for computers. The system is developed using Python 3.8+ and Flask, incorporating libraries such as Pandas, Numpy, TensorFlow or PyTorch, OpenCV, Matplotlib, and Seaborn for data processing, deep learning, and visualization.

Hardware Requirements :

- System Device : Mobile or Computer
- Processor : Intel Core i7 / AMD Ryzen or Higher
- RAM : 16GB or Higher (Both mobile and computer)
- Computer Storage : 512 GB SSD or Higher
- Mobile Storage : 64 GB or Higher
- Camera : 1080p Full HD and Minimum 30FPS

Software Requirements :

- Mobile OS : Android version 10
- Computer OS : Windows 10 or Higher
- Languages : Python 3.8+, Flask
- Libraries : Pandas, Numpy, TensorFlow or PyTorch, OpenCV, Matplotlib and Seaborn.

3.2 SYSTEM ARCHITECTURE

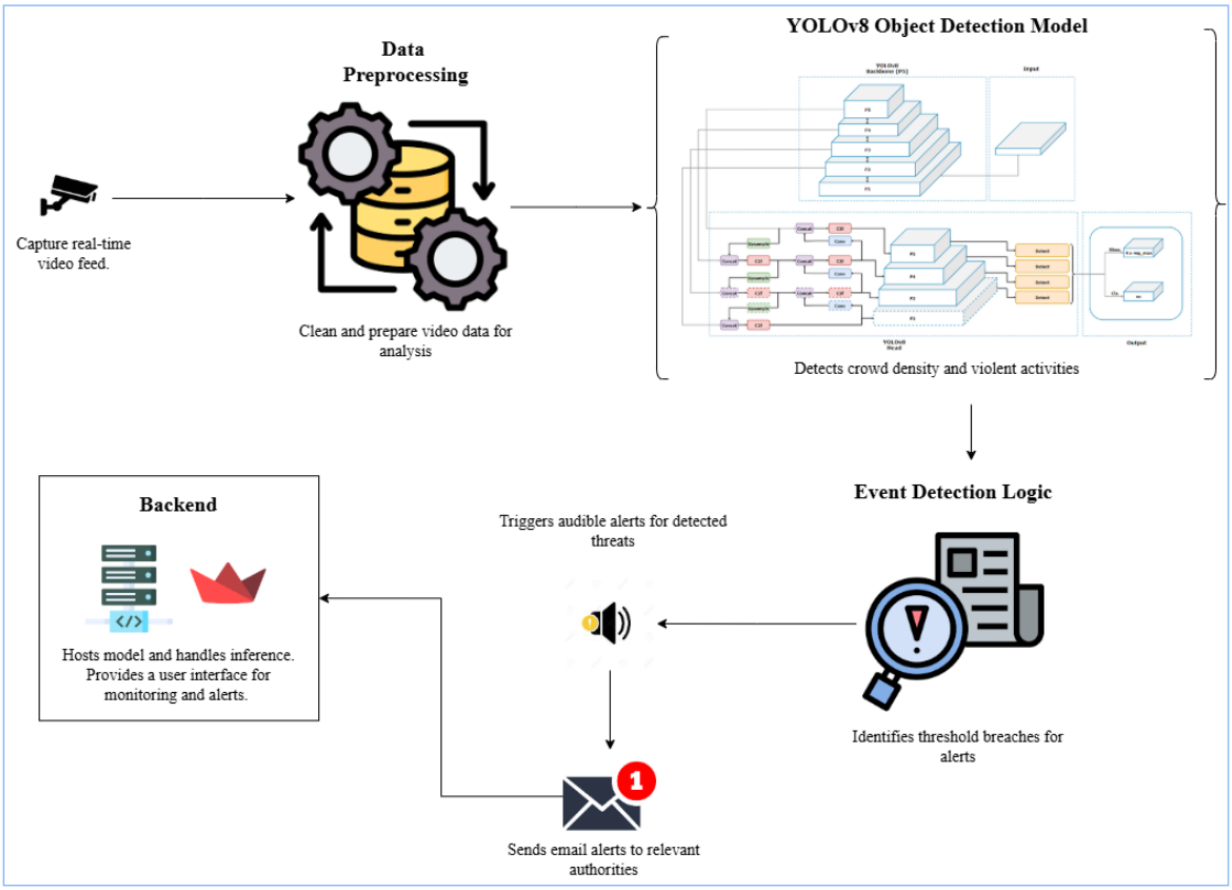


Fig No.1: System Architecture

1. Data Layer:

Handles storage and management of important system data, including:

Event Logs – Stores detected incidents for analysis.

Video Snapshots – Captures frames of suspicious activity.

System Configurations – Stores system setting and preferences.

2. Application Layer:

Responsible for the core functionalities, including:

Real-Time Video Processing – Analyzes live video feeds for activity detection.

YOLOv8-Based Violence Detection – Uses YOLOv8, an advanced object detection model, to identify violence in public spaces.

3. AI Model Layer:

The intelligence behind the system, including:

YOLOv8 Model Deployment – Executes deep learning-based violence detection.

TensorFlow & PyTorch – Frameworks used for training and running AI models.

4. Communication Layer:

Ensures immediate notifications and alerts in case of threats, including:

Email Alerts – Sends notifications to security personnel.

Sound Alarms – Triggers loud alarms for real-time intervention.

SMS Integration – Sends automated messages to relevant authorities.

5. Infrastructure Layer:

Defines how and where the system is deployed, ensuring scalability and high performance, including:

Cloud-Based Deployments – Enables remote access and scalability.

Edge Computing – Processes data locally for low-latency analysis.

GPU Acceleration – Uses high-performance GPUs for fast video analysis.

3.3 PROPOSED METHODOLOGY

To overcome the limitations of existing systems, this project proposes a real-time, AI-powered crowd and violence detection system using YOLOv8. The proposed system is designed to automate surveillance tasks, improve accuracy, and provide real-time alerts to ensure a faster response to security threats. The system leverages YOLOv8, one of the most advanced deep learning object detection models, for analyzing real-time video streams. Unlike traditional models, YOLOv8 can process multiple frames per second (FPS) while maintaining high accuracy. This ensures that even in high-traffic areas, the system can detect sudden crowd movements and violent actions without delays.

A key feature of the proposed system is its integration with an automated alert mechanism. Upon detecting a potential threat, the system triggers an audio alarm and sends email notifications to relevant authorities. This ensures that security personnel are immediately informed, allowing them to respond before an incident escalates. The proposed system also includes a user-friendly web-based dashboard, which allows security teams to monitor live feeds, receive alerts, and adjust system parameters in real time. This dashboard will be built using Flask and OpenCV, enabling seamless video streaming and alert management.

Another major improvement in the proposed system is the continuous learning approach. The model will be updated periodically with new data, ensuring that it adapts to changing patterns of violence and crowd behavior. This will reduce the false positive rate and increase the system's reliability over time.

3.3.1 DATABASE DESIGN

For the Real-Time Crowd and Violence Detection System, the database design will focus on storing relevant information such as video frames, detection results, alerts, and user data. Below is a basic design for your database schema.

Entities:

1. User - Stores user information for authentication.
2. VideoFrame - Stores metadata and the actual video frames being processed.
3. DetectionResult - Stores the results of the analysis, including whether violence or threats were detected.
4. Alert - Stores alert data triggered by detection, including timestamps and notification details.
5. Notification - Stores notifications sent to security personnel or users.

1. User Table

Column Name	Data Type	Description
user_id	INT	Primary Key, Auto Increment
username	VARCHAR(255)	Username for login (unique)
password_hash	VARCHAR(255)	Hashed password
role	ENUM('admin', 'security', 'user')	Role of the user (Admin, Security, or Regular User)
last_login	DATETIME	Timestamp of the last login

2.VideoFrame Table

Column Name	Data Type	Description
frame_id	INT	Primary Key, Auto Increment
video_source	VARCHAR(255)	Source of the video (e.g., camera ID or stream)
timestamp	DATETIME	Timestamp of the frame
frame_data	BLOB	Raw video frame data (image, encoded, etc.)
created_at	DATETIME	Timestamp when the frame was captured

3. DetectionResult Table

Column Name	Data Type	Description
detection_id	INT	Primary Key, Auto Increment
frame_id	INT	Foreign Key to VideoFrame (frame_id)
threat_detected	BOOLEAN	Whether a threat was detected (TRUE/FALSE)
violence_detected	BOOLEAN	Whether violence was detected (TRUE/FALSE)
analysis_time	DATETIME	Timestamp when analysis was performed
created_at	DATETIME	Timestamp when detection result was recorded

4. Alert Table

Column Name	Data Type	Description
alert_id	INT	Primary Key, Auto Increment
detection_id	INT	Foreign Key to DetectionResult (detection_id)
alert_type	ENUM('email', 'audio', 'both')	Type of alert (email, audio, or both)
alert_time	DATETIME	Timestamp when the alert was triggered
alert_status	ENUM('pending', 'sent', 'acknowledged')	Status of the alert (pending, sent, acknowledged)

5. Notification Table

Column Name	Data Type	Description
notification_id	INT	Primary Key, Auto Increment
alert_id	INT	Foreign Key to Alert (alert_id)
user_id	INT	Foreign Key to User (user_id)
notification_time	DATETIME	Timestamp when the notification was sent
notification_type	ENUM('email', 'sms', 'app')	Type of notification (email, SMS, app)

3.3.2 INPUT DESIGN

The input design ensures efficient data processing for real-time violence detection. It begins with data collection and preprocessing**,** where a well-labeled dataset of violent and non-violent actions is prepared. Data augmentation techniques like resizing normalization, rotation, flipping, and contrast adjustments enhance model accuracy. Noisy or redundant frames are removed for better performance.

The model training phase utilizes a CNN-based YOLOv8 architecture, optimized through transfer learning and batch processing. Hyperparameters like learning rate and confidence thresholds are fine-tuned for accuracy. Model performance is evaluated using Precision, Recall, and F1-Score.

For real-time detection, the YOLOv8 model is integrated with OpenCV, analyzing 30-45 FPS to detect aggressive movements and crowd surges. A confidence threshold of 0.85 reduces false positives, while GPU acceleration ensures scalability.

The alert mechanism includes sound alarms, email notifications with video snapshots, and future SMS alerts. Alerts are categorized by severity and stored in a logging system for future analysis.

A web-based dashboard built with Flask and HTML5 enables live monitoring, alert history tracking, and system customization. Role-based authentication ensures controlled access. The system workflow follows a structured process: capturing video, detecting suspicious activities, filtering false positives, triggering alerts, and updating the UI in real time. This streamlined input design enhances security monitoring and response efficiency.

3.3.3 MODULE DESIGN

DATA FLOW DIAGRAMS

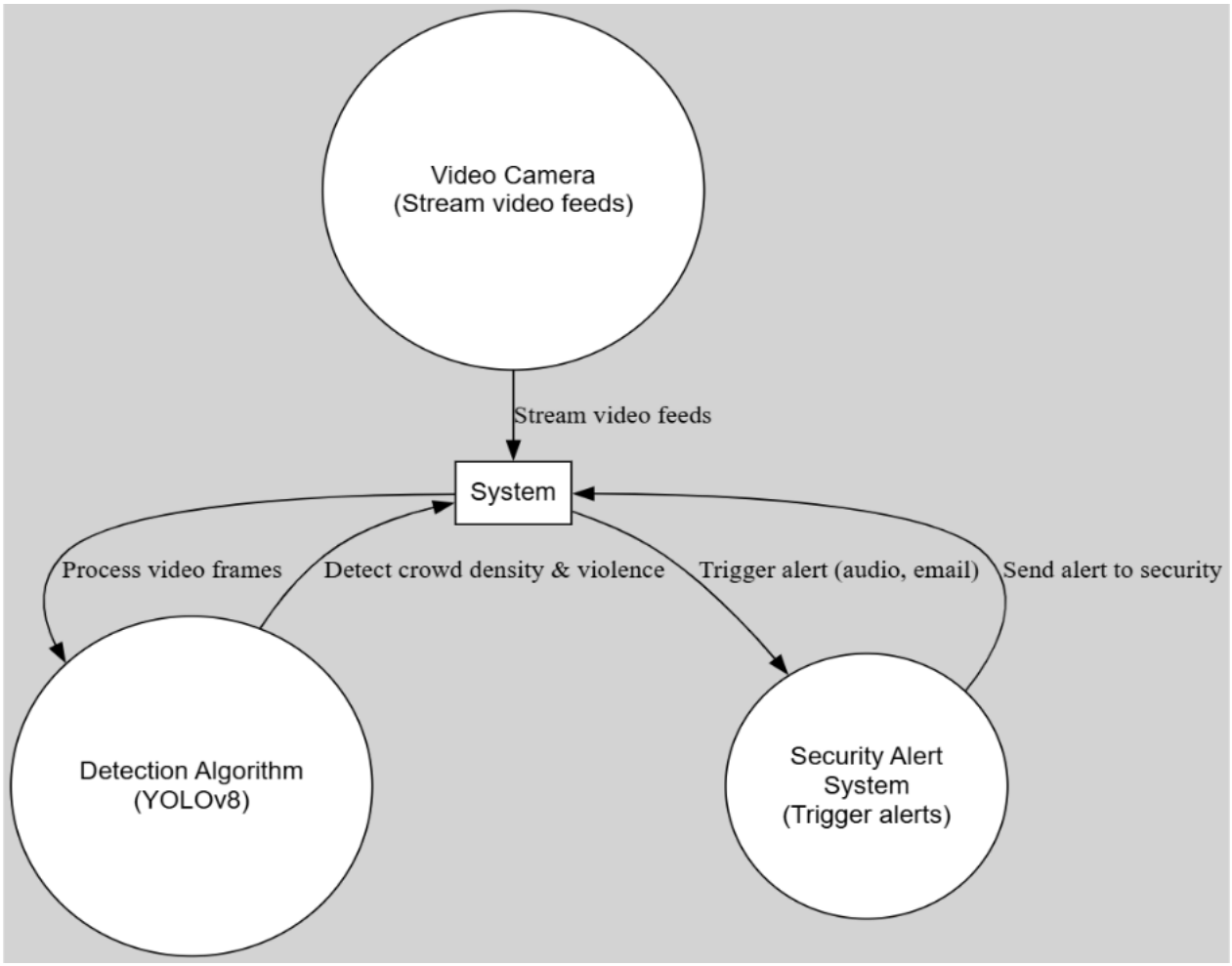


Fig no.2 Level 0 DFD

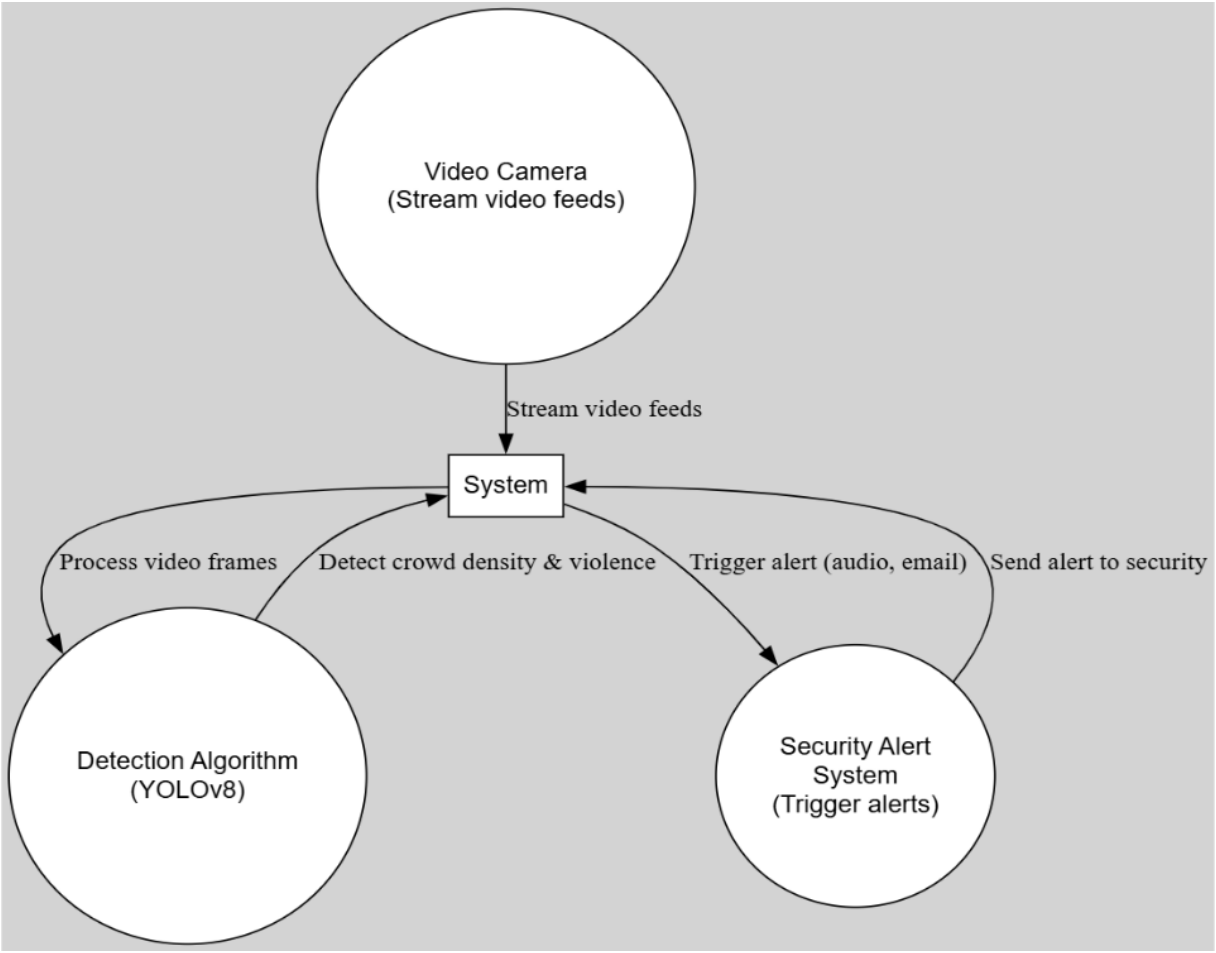


Fig no.3 Level 1 DFD

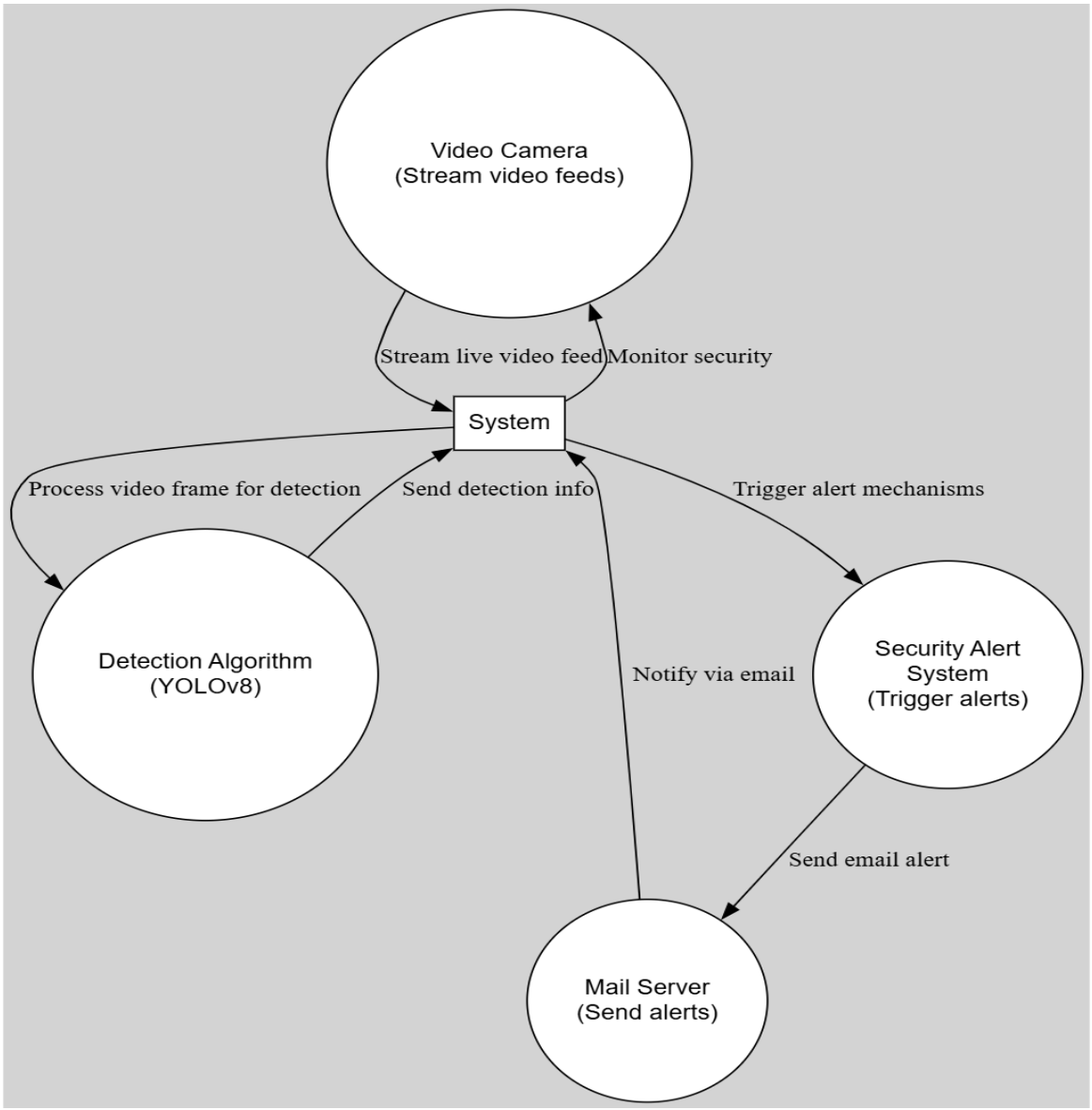


Fig no.4 Level 2 DFD

UML DIAGRAMS

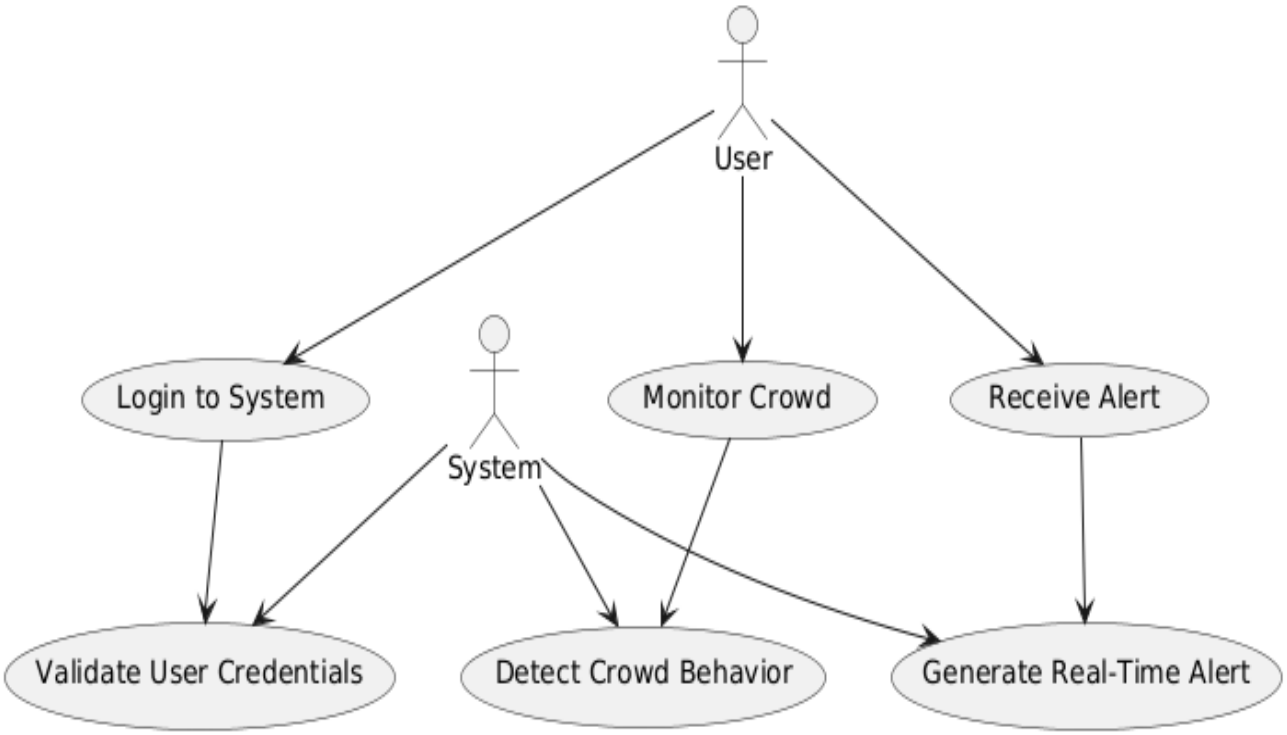


Fig no.5 Use Case Diagram

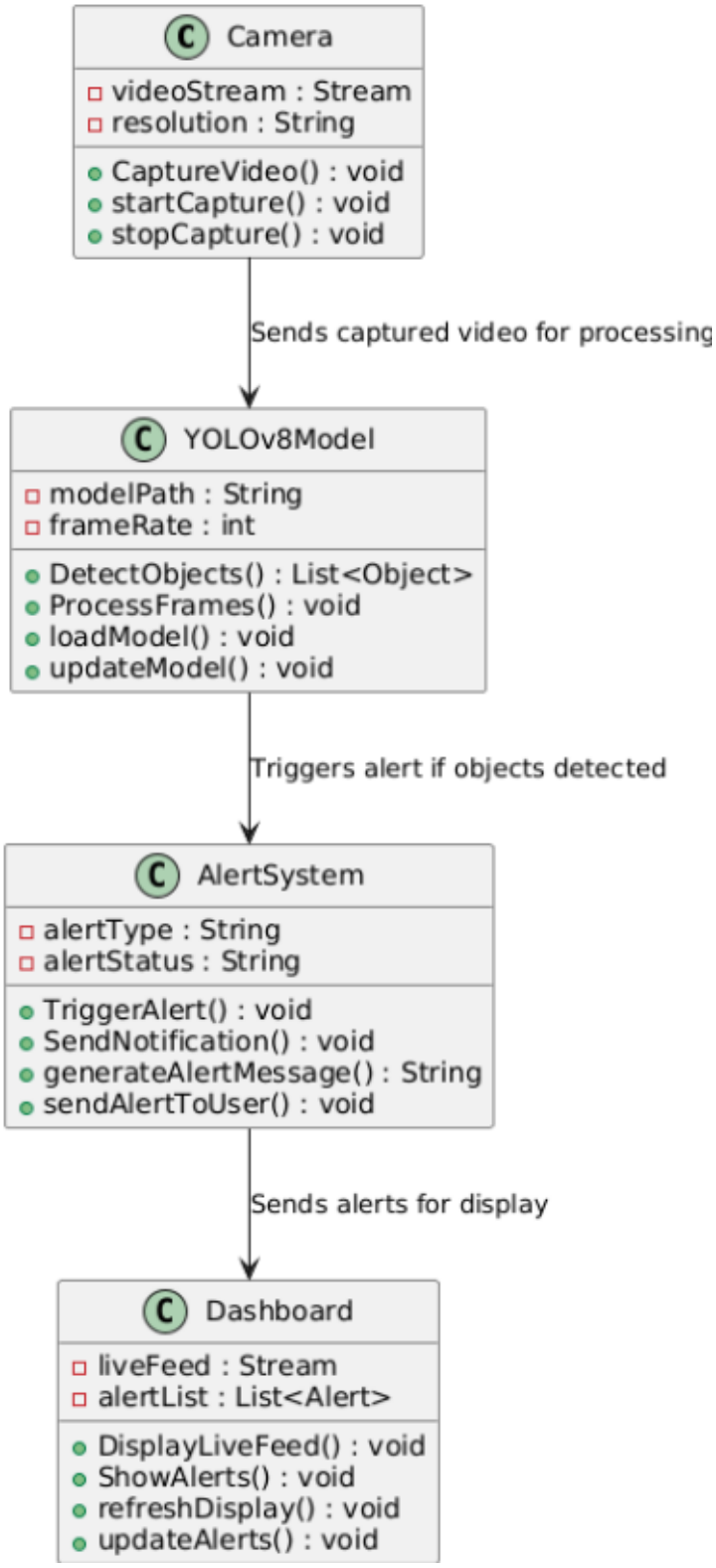


Fig no.6 Class Diagram

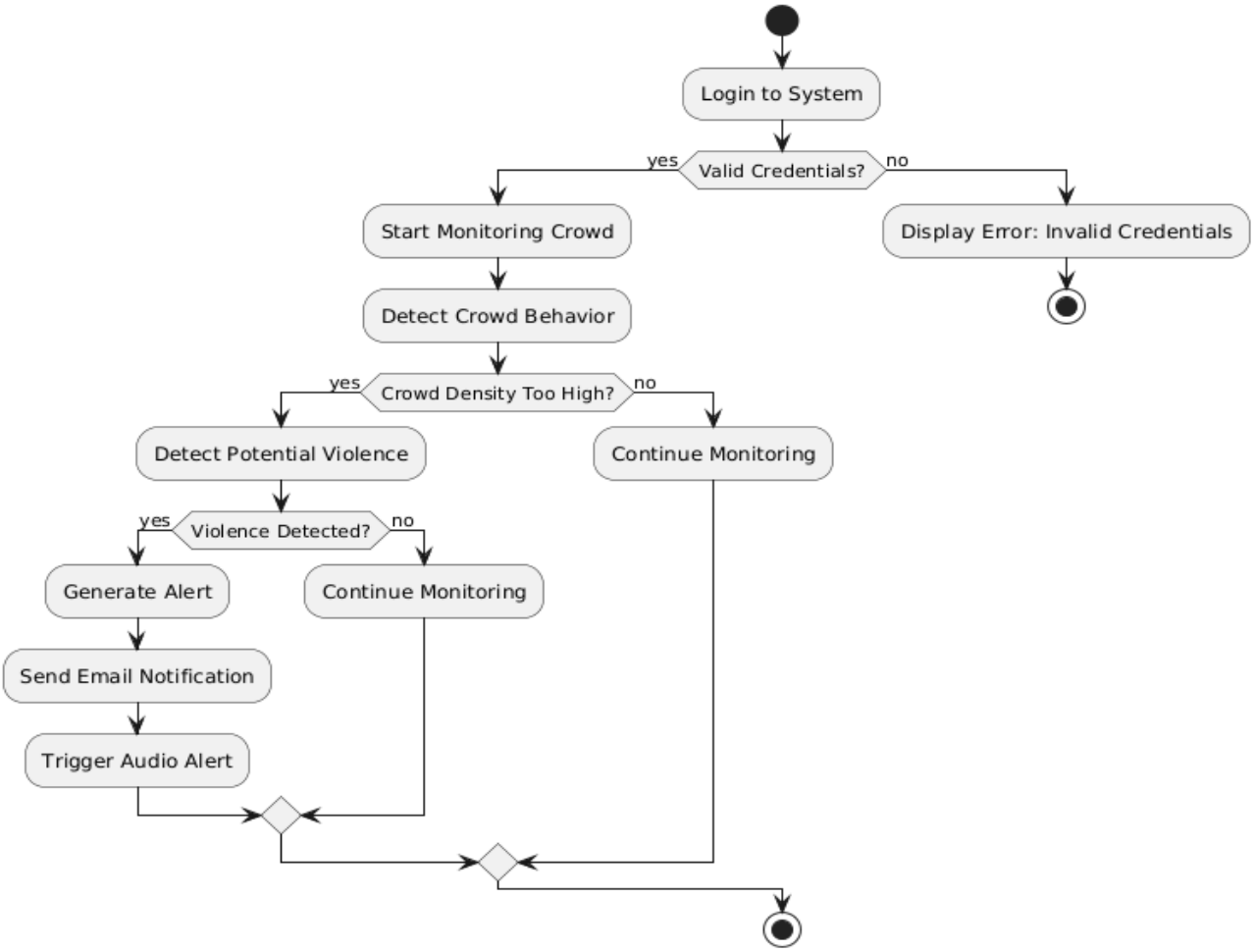


Fig no. 7 Activity Diagram

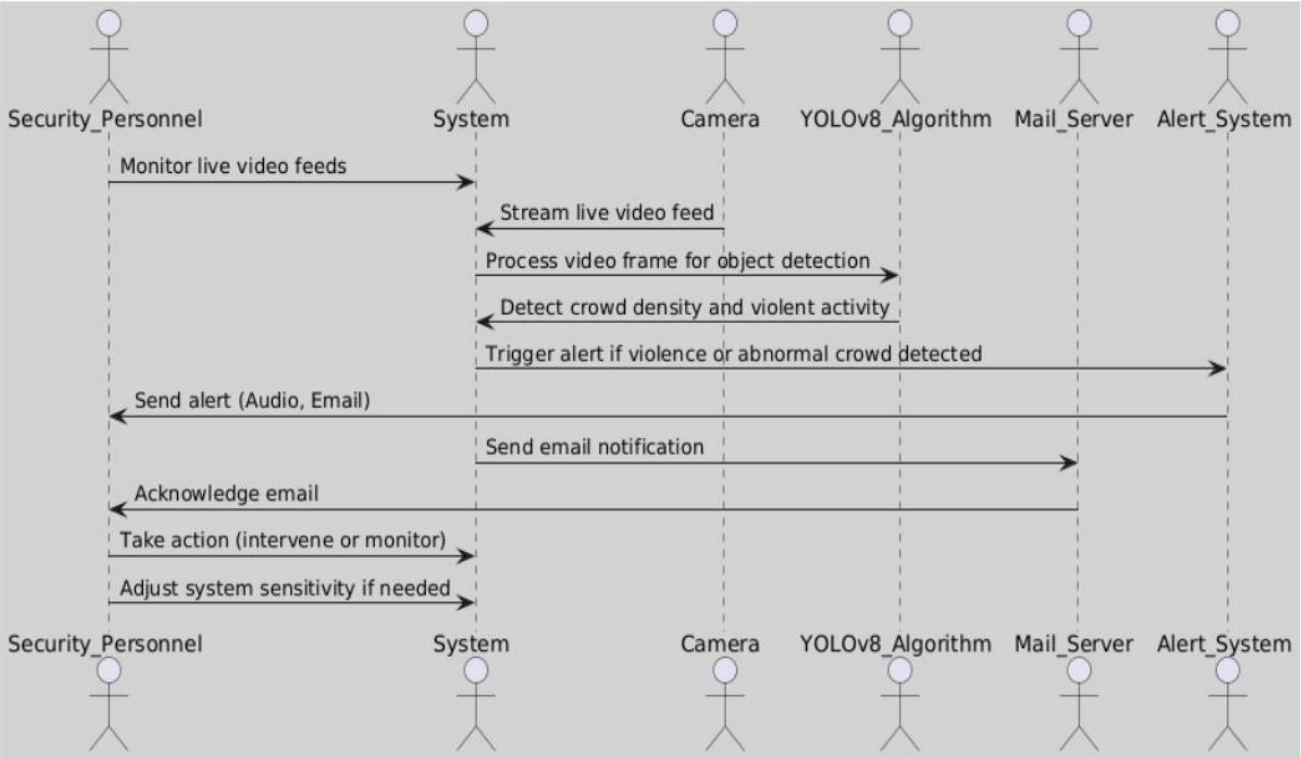


Fig no. 8 Sequence Diagram

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 REAL-TIME DETECTION AND INFERENCE

The trained YOLOv8 model is deployed in a real-time processing system that continuously analyzes live video feeds, ensuring rapid detection of violent activities. The system is designed for high accuracy, minimal latency, and scalability to handle multiple surveillance cameras simultaneously.

- Integration with OpenCV: The YOLOv8 model is integrated with OpenCV for efficient frame capture and real-time processing. OpenCV optimizes video feed handling by utilizing hardware acceleration and multi-threaded processing, ensuring minimal delay in detection.
- Frame-by-Frame Analysis: The system processes 30-45 frames per second (FPS)**,** allowing for smooth and uninterrupted real-time monitoring. Advanced buffering techniques help prevent frame drops, ensuring consistent detection.
- Violence Detection Logic: The model analyzes aggressive hand movements, body postures, rapid movements, and crowd surges to identify potentially violent behaviors. It leverages deep learning-based human pose estimation and object trajectory tracking to enhance accuracy.
- Filtering False Positives: A confidence threshold of 0.85 ensures only high-probability detections trigger alerts, reducing unnecessary alarms. The system employs additional filtering mechanisms, such as temporal consistency checks and motion analysis, to differentiate between actual violence and harmless activities like running or waving.
- Scalability: The detection pipeline is optimized for GPU acceleration**,** enabling deployment across large-scale security networks, such as airports, stadiums, and public events**.**

4.2 ALERT MECHANISM AND NOTIFICATION SYSTEM

The alert mechanism is a critical component of the system, ensuring that security personnel receive immediate notifications when violent activities are detected. The system employs multiple notification channels, allowing for a swift response to potential threats while minimizing false alarms through intelligent alert filtering.

- Upon detecting violent activities: When the system identifies potential violence, it instantly triggers an alert to notify security personnel. Detection is based on AI-driven anomaly detection, ensuring that only relevant threats activate alerts. The system can be integrated with surveillance command centers, mobile apps, or desktop dashboards, allowing security teams to take immediate action.
- Email Notification System: The system utilizes SMTP-based email alerts, automatically sending detailed reports to predefined recipients, such as security teams, administrators, or law enforcement agencies. Each email includes timestamped video snapshots, location details, and a brief summary of the detected incident.
- Multi-Tier Alert Levels: The system categorizes alerts based on severity, ensuring that high-risk threats receive immediate attention while low-priority detections do not overwhelm security teams. The alert levels range from Level 1 (suspicious movement) to Level 3 (confirmed violent activity). AI-driven adaptive alerting ensures that the system learns from past incidents and refines its response strategy over time.

4.3 USER INTERFACE AND DASHBOARD

The system features an interactive web-based dashboard designed for real-time monitoring, alert management, and system customization. This dashboard serves as the central hub for security personnel, allowing them to analyze live video feeds, respond to alerts, and configure system parameters.

- Frontend Development: The UI is built using Flask and HTML5, allowing seamless interaction with the backend model.Live Video Streaming: Security teams can view real-time feeds with detected threats marked using bounding boxes.
- Alert Logs and History: The dashboard maintains a searchable and filterable database of past alerts, enabling quick retrieval and analysis of previous incidents. Each alert entry includes timestamps, alert severity, detected behavior, video snapshots, and response actions taken.
- Role-Based Access: The system incorporates multi-tiered access control, ensuring that only authorized personnel can modify critical system settings. Administrators have full access to system configurations, while security personnel have view-only or limited control over specific functions.

CHAPTER 5

RESULT & DISCUSSION

5.1 TESTING

Software Testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding, Testing presents an interesting anomaly for the software engineer.

5.1.1 Testing Objectives

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case is one that has a probability of finding an undiscovered error.
3. A successful test is one that uncovers an undiscovered error.

5.1.2 Testing Principles

- All tests should be traceable to end user requirements.
- Tests should be planned long before testing begins.
- Testing should begin on a small scale and progress towards testing in large.
- Exhaustive testing is not possible.
- To be most effective testing should be conducted by a independent third party.

5.1.3 TEST CASES

S.NO	Test Cases	Test Procedure	Test Input	Expected Result	Actual Result
1	**Valid Crowd Detection**	- Enter a live video feed. - Apply YOLOv8 for crowd detection.	Video feed with a crowd of 30 people.	Crowd density should be calculated correctly.	Crowd density calculated as expected.
2	**Invalid Crowd Detection**	- Enter a live video feed with no people. - Apply YOLOv8 for crowd detection.	Video feed with no people.	No crowd density detected.	No crowd detected as expected.
3	**Valid Violence Detection**	- Enter a live video feed. - Apply YOLOv8 for violence detection.	Video feed showing a fight.	Violence should be detected with appropriate alert.	Violence detected, alert triggered.
4	**Invalid Violence Detection**	- Enter a live video feed. - Apply YOLOv8 for violence detection.	Video feed showing non-violent crowd.	No violence detected.	No violence detected as expected.
5	**Database Connection**	- Log in to the app. - Verify that the app communicates with the database.	Valid user credentials.	Successful connection to the database.	Successfully connected to the database.

6	**Invalid Database Connection**	- Modify the database credentials in app config. - Log in to the app.	Invalid database credentials.	Unauthorized access should be shown.	Error message shown for unauthorized access.
7	**Real-time Detection**	- Enter live video stream. - Trigger crowd or violence detection.	Video feed with multiple events.	Detection should trigger alerts in real-time.	Alerts triggered as expected.
8	**Alert Notification**	- Detect violence or crowd density. - Send email notifications.	Trigger violence detection.	Email notification should be sent.	Email sent successfully.
9	**System Performance**	- Process continuous video feed. - Measure FPS.	Continuous live video feed.	FPS should be above 25 for smooth detection.	FPS maintained at 30+.

5.2 RESULT

The main objective of the real-time crowd and violence detection system using YOLOv8 is to provide an efficient, automated solution for identifying and alerting security personnel to potentially violent incidents in public spaces. By leveraging YOLOv8's advanced object detection capabilities, the system processes live video feeds in real-time, detecting aggressive behaviors, such as hand movements and body postures, and crowd surges with high accuracy. The system is designed to be highly scalable, making it suitable for large-scale deployments in environments like shopping malls, stadiums, and transportation hubs. With a GPU-optimized pipeline, the system can handle high frame rates, ensuring smooth and low-latency detection. Alerts are triggered through audio alarms and email notifications with timestamped snapshots, enabling immediate responses from security teams. The system's user-friendly dashboard provides live video streaming with detected threats marked, along with searchable alert logs for investigation. This solution not only enhances public safety but also improves operational efficiency by reducing false positives with adjustable sensitivity settings. It provides a reliable, real-time security monitoring system that adapts to various environments and integrates seamlessly with existing security infrastructure.

CHAPTER 6
CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

The real-time crowd and violence detection system using YOLOv8 represents a major advancement in AI-powered surveillance, offering automated real-time detection and alert mechanisms to enhance public safety and security operations. By leveraging deep learning and computer vision, the system efficiently identifies sudden crowd density changes and violent activities, ensuring rapid response by security personnel. The integration of real-time notifications, sound alarms, and an interactive monitoring dashboard minimizes the reliance on manual surveillance, reducing response time and improving situational awareness. With high accuracy and processing speed, the system is adaptable for deployment in high-risk areas such as public events, transportation hubs, and commercial spaces.

6.2 FUTURE SCOPE

Future enhancements will focus on integrating audio-based violence detection to identify gunshots, screams, and distress signals, improving accuracy through multimodal analysis. Additionally, edge computing and IoT-enabled real-time processing will allow deployment on low-power devices, making the system more scalable for smart city surveillance networks. Further improvements in predictive analytics, AI-driven behavioral analysis, and multi-camera synchronization will enhance threat anticipation and coordinated security responses. Cloud-based deployment on platforms like AWS and Google Cloud will enable remote access for law enforcement agencies, ensuring faster decision-making and incident response. With these advancements, the system has the potential to revolutionize AI-driven security monitoring, providing safer public spaces and proactive crime prevention.

APPENDICES

APPENDIX 1: SDG GOALS

Real-time crowd and violence detection can contribute to achieving Sustainable Development Goals (SDG) . Reduced Inequalities in several ways. Let's explore how:

1. **SDG 11: Sustainable Cities and Communities**

   Real-time crowd and violence detection directly contributes to making cities and communities safer, more resilient, and inclusive. By implementing real-time monitoring for crowd density and violence detection, it enhances public safety in urban areas, public gatherings, and events. Timely alerts enable authorities to act quickly, reducing the risk of harm and ensuring safer environments for all community members.

2. **SDG 16: Peace, Justice, and Strong Institutions**

   The project promotes peaceful and inclusive societies by providing a tool to detect violent activities and crowd-related incidents. By offering real-time alerts and notifications, the system aids law enforcement and security agencies in maintaining peace and order. This contributes to strengthening institutions by enabling efficient, data-driven responses to potential security threats.

3. **SDG 9: Industry, Innovation, and Infrastructure**

   The integration of advanced technologies like YOLOv8, deep learning frameworks (TensorFlow, PyTorch), and computer vision (OpenCV) positions this project as an innovative solution to address public safety concerns. It promotes the use of cutting-edge technology to enhance infrastructure resilience and improve response systems in public safety management.

4. **SDG 3: Good Health and Well-Being**

   By improving public safety and enabling faster responses to violent events, the system indirectly supports the well-being of individuals in public spaces. A safer environment promotes mental and physical health by reducing the risk of harm and violence, ensuring that people can freely participate in events and public spaces without fear.

APPENDIX 2: SOURCE CODE

app2.py :

```
# Python In-built packages
from pathlib import Path
import PIL
import supervision as sv
from pydub.playback import play
from playsound import playsound
import streamlit as st
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
import time
from collections import defaultdict
import cv2
from PIL import Image
import numpy as np

def play_sound():
    """Play system beep sound"""
    try:
        import winsound
        winsound.Beep(1000, 500)  # frequency=1000Hz, duration=500ms
    except Exception as e:
        st.error(f"Error playing sound: {str(e)}")

# Initialize session state
if 'processing_complete' not in st.session_state:
    st.session_state.processing_complete = False
if 'final_results' not in st.session_state:
    st.session_state.final_results = None
if 'video_processed_frames' not in st.session_state:
    st.session_state.video_processed_frames = 0
if 'total_frames' not in st.session_state:
    st.session_state.total_frames = 0

my_email = "vivekraina33.vr@gmail.com"
password_key = "xsfuoajtzwfqhnvl"
gmail_server = "smtp.gmail.com"
gmail_port = 587
```

```
# Email setup code...
my_server = smtplib.SMTP(gmail_server, gmail_port)
my_server.ehlo()
my_server.starttls()
my_server.login(my_email, password_key)
message = MIMEMultipart("alternative")
text_content = "Alert: Crowd/Violence Detection Alert"
message.attach(MIMEText(text_content))
recruiter_email = "ayushtiwari.creatorslab@gmail.com"
msg_string = message.as_string()

import settings
import helper

class CrowdAnalytics:
    def __init__(self):
        self.reset()

    def reset(self):
        self.people_counts = []
        self.violence_detections = []
        self.alert_triggered = False
        self.max_people_count = 0
        self.highest_density = "Very Low"

    def update(self, count=0, detections=None):
        if count > 0:
            self.people_counts.append(count)
            self.max_people_count = max(self.max_people_count, count)
            self._update_density(count)

        if detections:
            if isinstance(detections, list):
                self.violence_detections.extend([d for d in detections if d in ["Violence", "NonViolence"]])

    def _update_density(self, count):
        if count >= 50:
            self.highest_density = "High"
        elif count >= 10:
            if self.highest_density not in ["High"]:
                self.highest_density = "Medium"
        elif count >= 1:
            if self.highest_density not in ["High", "Medium"]:
```

```
            self.highest_density = "Low"

    def get_final_stats(self):
        stats = {
            'max_people': self.max_people_count,
            'density': self.highest_density,
            'violence_detected': "Violence" in self.violence_detections
        }
        return stats


def process_video(video_path, model, model_name, confidence, analytics, progress_bar, status_text):
    """Process video and return final statistics"""
    try:
        cap = cv2.VideoCapture(video_path)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        st.session_state.total_frames = total_frames

        if total_frames <= 0:
            st.error("Could not read video frames")
            return None

        frame_placeholder = st.empty()
        processed_frames = 0

        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            processed_frames += 1
            progress = int((processed_frames / total_frames) * 100)
            progress_bar.progress(progress)
            status_text.text(f"Processing frame {processed_frames}/{total_frames}")


            # Update session state
            st.session_state.video_processed_frames = processed_frames

        cap.release()
        return analytics.get_final_stats()

    except Exception as e:
```

```
        st.error(f"Video processing error: {str(e)}")
        if 'cap' in locals():
            cap.release()
        return None

# Main Streamlit UI
st.set_page_config(
    page_title="Crowd and Violence Detection",
    page_icon="🤖",
    layout="wide"
)

st.title("Crowd and Violence Detection")

# Initialize analytics
crowd_analytics = CrowdAnalytics()

# Sidebar configuration
st.sidebar.header("Model Config")

model_type = st.sidebar.radio(
    "Select Task", ['Crowd Detection', 'Violence Detection'])

confidence = float(st.sidebar.slider(
    "Select Model Confidence", 1, 100, 40)) / 100

# Model selection
if model_type == 'Crowd Detection':
    model_path = Path(settings.DETECTION_MODEL1)
    model_name = "crowd"
elif model_type == 'Violence Detection':
    model_path = Path(settings.DETECTION_MODEL2)
    model_name = "violenceguns"

# Load model
try:
    if model_name == "crowd":
        model = helper.load_model(model_path)
    elif model_name == "violenceguns":
        model = helper.load_model_vguns(model_path)
except Exception as ex:
    st.error(f"Unable to load model. Check the specified path: {model_path}")
    st.error(ex)
```

```
# Source selection
st.sidebar.header("Image/Video Config")
source_radio = st.sidebar.radio("Source", settings.SOURCES_LIST)

if source_radio == settings.IMAGE:
    source_img = st.sidebar.file_uploader(
        "Choose an image...", type=("jpg", "jpeg", "png", 'bmp', 'webp'))

    col1, col2 = st.columns(2)

    with col1:
        try:
            if source_img is None:
                default_image_path = str(settings.DEFAULT_IMAGE)
                default_image = PIL.Image.open(default_image_path)
                st.image(default_image_path, caption="Default Image",
                         use_column_width=True)
            else:
                uploaded_image = PIL.Image.open(source_img)
                st.image(source_img, caption="Uploaded Image",
                         use_column_width=True)
        except Exception as ex:
            st.error("Error occurred while opening the image.")
            st.error(ex)

    with col2:
        if source_img is None:
            default_detected_image_path = str(settings.DEFAULT_DETECT_IMAGE)
            default_detected_image = PIL.Image.open(
                default_detected_image_path)
            st.image(default_detected_image_path, caption='Detected Image',
                     use_column_width=True)
        else:
            if st.sidebar.button('Detect'):
                crowd_analytics.reset()

                if model_name == "violenceguns":
                    res = model.infer(uploaded_image,
                                      confidence=confidence)
                    detections = sv.Detections.from_inference(res[0].dict(by_alias=True,
exclude_none=True))
                    bounding_box_annotator = sv.RoundBoxAnnotator()
                    label_annotator = sv.LabelAnnotator()
```

```
        annotated_image = bounding_box_annotator.annotate(
            scene=uploaded_image, detections=detections)
        annotated_image = label_annotator.annotate(
            scene=annotated_image, detections=detections)
        st.image(annotated_image, caption='Detected Image',
                 use_column_width=True)

        class_names = detections.data['class_name']
        threat_detected = False

        for class_name in class_names:
            if class_name == "Violence":
                threat_detected = True
                st.error("Violence Detected!")
                play_sound()
                st.warning("Alert Sound Played 🚨 ")
                my_server.sendmail(from_addr=my_email,
                                   to_addrs=recruiter_email,
                                   msg=msg_string)
                st.success("Alert Email Sent")

        if not threat_detected:
            st.success("No Violence Detected")

    elif model_name == "crowd":
        res = model.predict(uploaded_image,
                            conf=confidence
                            )
        names = res[0].names
        class_detections_values = []
        for k, v in names.items():
            class_detections_values.append(res[0].boxes.cls.tolist().count(k))
        classes_detected = dict(zip(names.values(), class_detections_values))

        res_plotted = res[0].plot()[:, :, ::-1]
        st.image(res_plotted, caption='Detected Image',
                 use_column_width=True)

        if 'people' in classes_detected:
            no_of_people = classes_detected['people']
            if no_of_people:
                if no_of_people == 0:
                    st.info("Crowd Density: Very Low")
```

```
                    st.success("No of people: {}".format(no_of_people))
                elif 1< no_of_people < 10:
                    st.info("Crowd Density: Low")
                    st.success("No of people: {}".format(no_of_people))
                elif 10< no_of_people <50:
                    st.info("Crowd Density: Medium")
                    st.success("No of people: {}".format(no_of_people))
                    play_sound()
                    st.warning("Alert Sound Played 🚨 ")
                if no_of_people > 50:
                    st.title("Crowd Density: High")
                    st.title("No of people: {}".format(no_of_people))
                    play_sound()
                    my_server.sendmail(from_addr=my_email,
                                       to_addrs=recruiter_email,
                                       msg=msg_string)
                    st.success("Alert Email Sent")
            else:
                st.title("No People")

elif source_radio == settings.VIDEO:
    uploaded_video = st.sidebar.file_uploader("Upload Video", type=['mp4', 'avi', 'mov'])

    if uploaded_video is not None:
        if st.sidebar.button('Process Video'):
            # Reset analytics
            crowd_analytics.reset()
            st.session_state.processing_complete = False

            # Create progress elements
            progress_bar = st.progress(0)
            status_text = st.empty()

            try:
                # Save uploaded video temporarily
                video_path = f"temp_video_{int(time.time())}.mp4"
                with open(video_path, 'wb') as f:
                    f.write(uploaded_video.read())

                # Process video
                final_stats = process_video(
                    video_path, model, model_name, confidence,
                    crowd_analytics, progress_bar, status_text
```

```
            )

            # Show final results
            st.success("Video Processing Complete!")

            with st.container():
                st.header("Final Analysis Results")

                if model_name == "crowd":
                    st.info(f"Maximum Crowd Density: {final_stats['density']}")
                    st.success(f"Peak People Count: {final_stats['max_people']}")


                elif model_name == "violenceguns":
                    if final_stats['violence_detected']:
                        st.error("⚠ Violence Detected in Video!")
                        play_sound()
                        my_server.sendmail(from_addr=my_email,
                                           to_addrs=recruiter_email,
                                           msg=msg_string)
                        st.success("Alert Email Sent")
                    else:
                        st.success("☑ No threats detected in video")

                # Cleanup
                import os
                os.remove(video_path)

            except Exception as ex:
                st.error(f"Error processing video: {str(ex)}")
            finally:
                progress_bar.empty()
                status_text.empty()
                st.session_state.processing_complete = True

elif source_radio == settings.WEBCAM:
    if st.sidebar.button('Start Webcam'):
        st.info("Webcam processing will start... (Implementation similar to video)")

elif source_radio == settings.RTSP:
    rtsp_url = st.sidebar.text_input("RTSP URL")
    if st.sidebar.button('Start RTSP Stream'):
        st.info("RTSP stream processing will start... (Implementation similar to video)")
```

```
elif source_radio == settings.YOUTUBE:
    youtube_url = st.sidebar.text_input("YouTube URL")
    if st.sidebar.button('Process YouTube Video'):
        st.info("YouTube video processing will start... (Implementation similar to video)")

else:
    st.error("Please select a valid source type!")

# Cleanup at the end
if st.session_state.processing_complete:
    my_server.quit()  # Clean server shutdown
```

helper.py :

```
from pathlib import Path
import sys

# Get the absolute path of the current file
file_path = Path(__file__).resolve()

# Get the parent directory of the current file
root_path = file_path.parent

# Add the root path to the sys.path list if it is not already there
if root_path not in sys.path:
    sys.path.append(str(root_path))

# Get the relative path of the root directory with respect to the current working directory
ROOT = root_path.relative_to(Path.cwd())

# Sources
IMAGE = 'Image'
VIDEO = 'Video'
WEBCAM = 'Webcam'
RTSP = 'RTSP'
YOUTUBE = 'YouTube'

SOURCES_LIST = [IMAGE, VIDEO]

# Images config
IMAGES_DIR = ROOT / 'images'
DEFAULT_IMAGE = IMAGES_DIR / 'office_4.jpg'
DEFAULT_DETECT_IMAGE = IMAGES_DIR / 'office_4_detected.jpg'
```

```
# Videos config
VIDEO_DIR = ROOT / 'videos'
VIDEO_1_PATH = VIDEO_DIR / 'people.mp4'
VIDEO_2_PATH = VIDEO_DIR / 'voilence.mp4'
VIDEO_3_PATH = VIDEO_DIR / 'video_3.mp4'
VIDEOS_DICT = {
   'video_1': VIDEO_1_PATH,
   'video_2': VIDEO_2_PATH,
   'video_3': VIDEO_3_PATH,
}

# ML Model config
MODEL_DIR = ROOT / 'weights'
DETECTION_MODEL1 = MODEL_DIR / 'bestcrowd.pt'
DETECTION_MODEL2 = MODEL_DIR / 'bestvoilence.pt'
DETECTION_MODEL3 = MODEL_DIR / 'bestvoilence.pt'

# In case of your custome model comment out the line above and
# Place your custom model pt file name at the line below
# DETECTION_MODEL = MODEL_DIR / 'my_detection_model.pt'

SEGMENTATION_MODEL = MODEL_DIR / 'yolov8n-seg.pt'

# Webcam
WEBCAM_PATH = 0
```

settings.py :

```
from ultralytics import YOLO
import time
import streamlit as st
import cv2
from pytube import YouTube
from playsound import playsound
from inference import get_model


import settings

def play_sound():
   sound_file = 'mixkit-classic-alarm-995.wav'  # Replace with the path to your sound file
   playsound(sound_file)

def load_model(model_path):
```

```
    """
    Loads a YOLO object detection model from the specified model_path.

    Parameters:
        model_path (str): The path to the YOLO model file.

    Returns:
        A YOLO object detection model.
    """
    model = YOLO(model_path)
    return model
def load_model_vguns(model_path):
    model = get_model(model_id="violence-weapon-detection/1",api_key="eSzt9jqUwL3SzdHp0dmr")
    return model


def display_tracker_options():
    display_tracker = st.radio("Display Tracker", ('Yes', 'No'))
    is_display_tracker = True if display_tracker == 'Yes' else False
    if is_display_tracker:
        tracker_type = st.radio("Tracker", ("bytetrack.yaml", "botsort.yaml"))
        return is_display_tracker, tracker_type
    return is_display_tracker, None


def _display_detected_frames(conf, model, st_frame, image, is_display_tracking=None,
tracker=None):
    """
    Display the detected objects on a video frame using the YOLOv8 model.

    Args:
    - conf (float): Confidence threshold for object detection.
    - model (YoloV8): A YOLOv8 object detection model.
    - st_frame (Streamlit object): A Streamlit object to display the detected video.
    - image (numpy array): A numpy array representing the video frame.
    - is_display_tracking (bool): A flag indicating whether to display object tracking (default=None).

    Returns:
    None
    """

    # Resize the image to a standard size
    image = cv2.resize(image, (720, int(720*(9/16))))
```

```
# Display object tracking, if specified
if is_display_tracking:
    res = model.track(image, conf=conf, persist=True, tracker=tracker)
    names = res[0].names
    class_detections_values = []
    for k, v in names.items():
        class_detections_values.append(res[0].boxes.cls.tolist().count(k))
    # create dictionary of objects detected per class
    classes_detected = dict(zip(names.values(), class_detections_values))
    if 'people' in classes_detected:
        no_of_people = classes_detected['people']
        #st.title(f"No of People: {no_of_people}")
        if no_of_people:
            if no_of_people == 0:
                st.markdown("Crowd Density: Very Low")
                st.markdown("No of people: {}".format(no_of_people))
            elif 1<= no_of_people < 10:
                st.markdown("Crowd Density: Low")
                st.markdown("No of people: {}".format(no_of_people))
            elif 10< no_of_people <50:
                st.markdown("Crowd Density: Medium")
                st.markdown("No of people: {}".format(no_of_people))
                play_sound()
                st.warning("Alert Sound Played 🚨")
            if no_of_people > 50:
                st.markdown("Crowd Density: High")
                st.markdown("No of people: {}".format(no_of_people))
                play_sound()
        else:
            st.title("No People")
    elif 'violence' in classes_detected:
        st.markdown("Violence Detected")
        play_sound()
        st.warning("Alert Sound Played 🚨")

else:
    # Predict the objects in the image using the YOLOv8 model
    res = model.predict(image, conf=conf)
    names = res[0].names
    class_detections_values = []
    for k, v in names.items():
        class_detections_values.append(res[0].boxes.cls.tolist().count(k))
    # create dictionary of objects detected per class
```

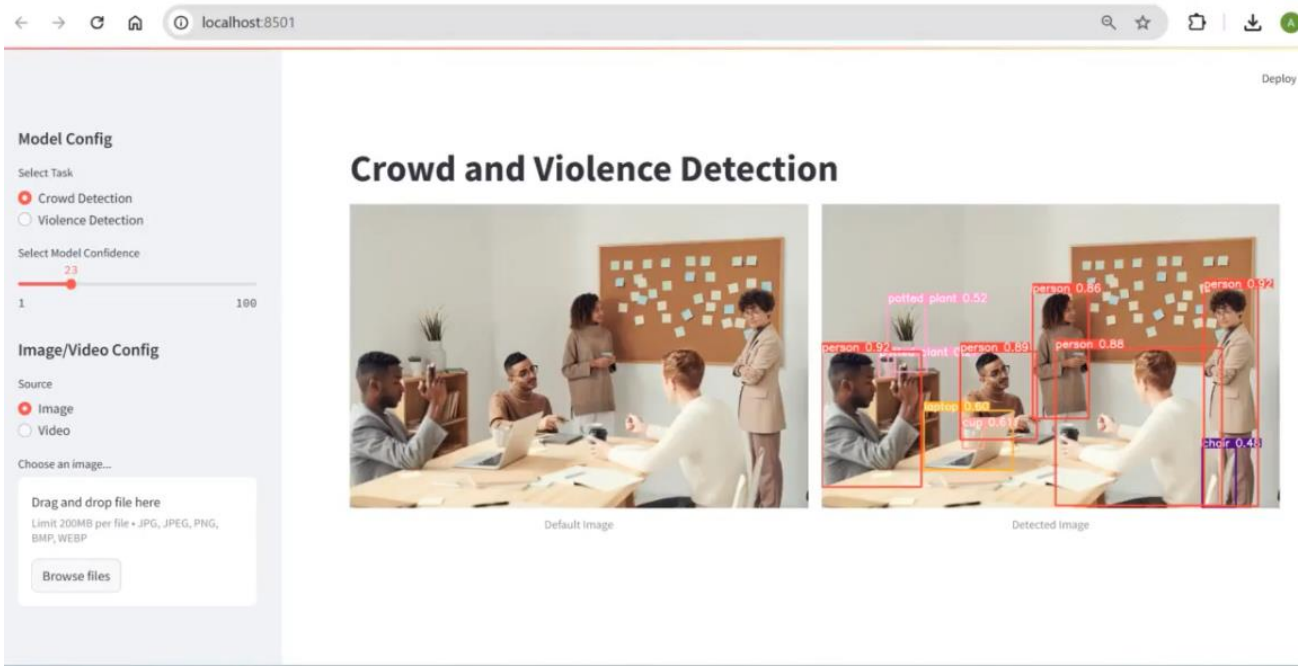
```
        classes_detected = dict(zip(names.values(), class_detections_values))

    # # Plot the detected objects on the video frame
    res_plotted = res[0].plot()
    st_frame.image(res_plotted,
                   caption='Detected Video',
                   channels="BGR",
                   use_column_width=True
                   )


def play_youtube_video(conf, model):
    """
    Plays a webcam stream. Detects Objects in real-time using the YOLOv8 object detection model.

    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the YOLOv8 model.

    Returns:
        None

    Raises:
        None
    """
    source_youtube = st.sidebar.text_input("YouTube Video url")

    is_display_tracker, tracker = display_tracker_options()

    if st.sidebar.button('Detect Objects'):
        try:
            yt = YouTube(source_youtube)
            stream = yt.streams.filter(file_extension="mp4", res=720).first()
            vid_cap = cv2.VideoCapture(stream.url)

            st_frame = st.empty()
            while (vid_cap.isOpened()):
                success, image = vid_cap.read()
                if success:
                    _display_detected_frames(conf,
                                             model,
                                             st_frame,
                                             image )
```

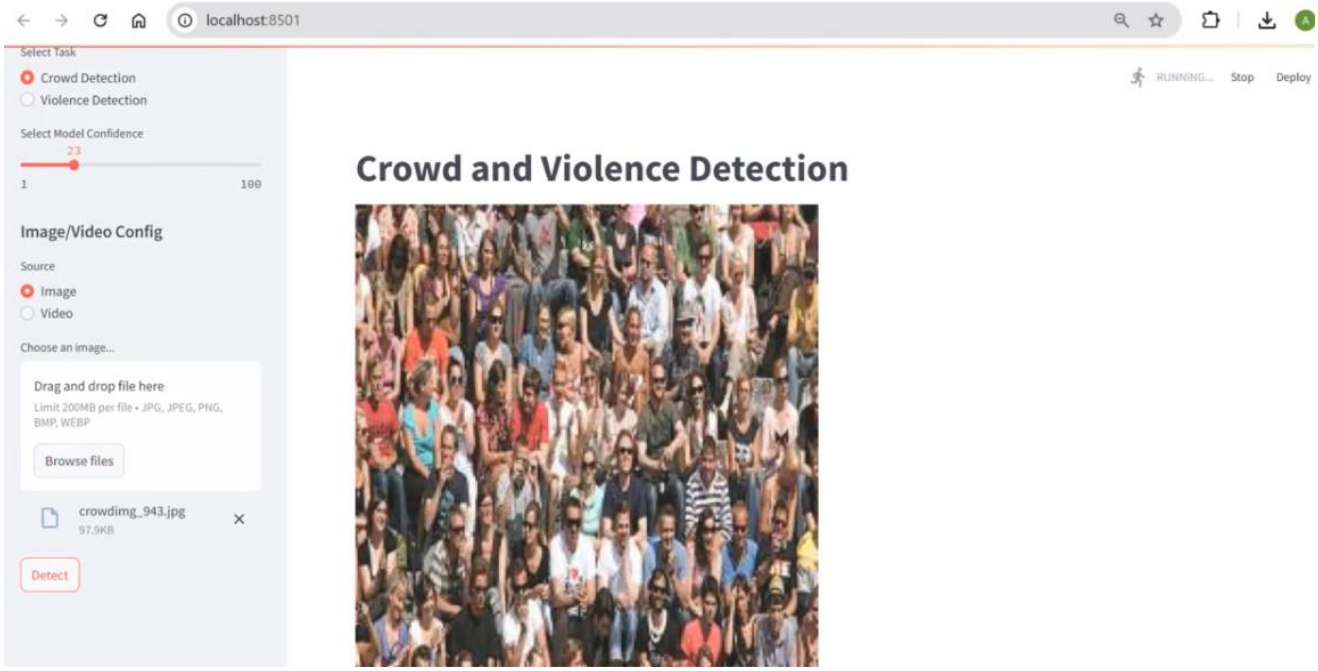
```
            else:
                vid_cap.release()
                break
        except Exception as e:
            st.sidebar.error("Error loading video: " + str(e))


def play_rtsp_stream(conf, model):
    """
    Plays an rtsp stream. Detects Objects in real-time using the YOLOv8 object detection model.

    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the YOLOv8 model.

    Returns:
        None

    Raises:
        None
    """
    source_rtsp = st.sidebar.text_input("rtsp stream url:")
    st.sidebar.caption('Example URL: rtsp://admin:12345@192.168.1.210:554/Streaming/Channels/101')
    is_display_tracker, tracker = display_tracker_options()
    if st.sidebar.button('Detect Objects'):
        try:
            vid_cap = cv2.VideoCapture(source_rtsp)
            st_frame = st.empty()
            while (vid_cap.isOpened()):
                success, image = vid_cap.read()
                if success:
                    _display_detected_frames(conf,
                                             model,
                                             st_frame,
                                             image,
                                             is_display_tracker,
                                             tracker
                                             )
                else:
                    vid_cap.release()
                    # vid_cap = cv2.VideoCapture(source_rtsp)
                    # time.sleep(0.1)
                    # continue
```

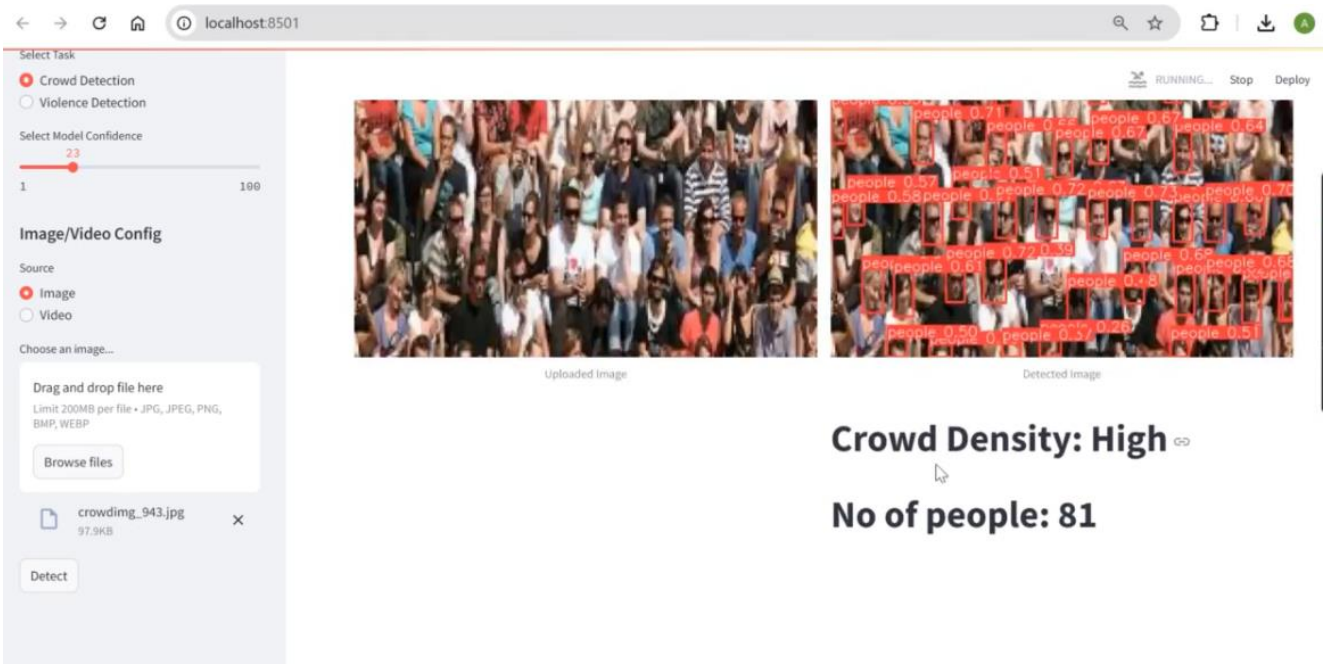
```
                break
        except Exception as e:
            vid_cap.release()
            st.sidebar.error("Error loading RTSP stream: " + str(e))


def play_webcam(conf, model):
    """
    Plays a webcam stream. Detects Objects in real-time using the YOLOv8 object detection model.

    Parameters:
        conf: Confidence of YOLOv8 model.
        model: An instance of the `YOLOv8` class containing the YOLOv8 model.

    Returns:
        None

    Raises:
        None
    """
    source_webcam = settings.WEBCAM_PATH
    is_display_tracker, tracker = display_tracker_options()
    if st.sidebar.button('Detect Objects'):
        try:
            vid_cap = cv2.VideoCapture(source_webcam)
            st_frame = st.empty()
            while (vid_cap.isOpened()):
                with st.container():
                    success, image = vid_cap.read()
                    if success:
                        _display_detected_frames(conf,
                                                 model,
                                                 st_frame,
                                                 image,
                                                 is_display_tracker,
                                                 tracker,
                                                 )
                    else:
                        vid_cap.release()
                        break
        except Exception as e:
            st.sidebar.error("Error loading video: " + str(e))
```

APPENDIX 3: SCREENSHOTS

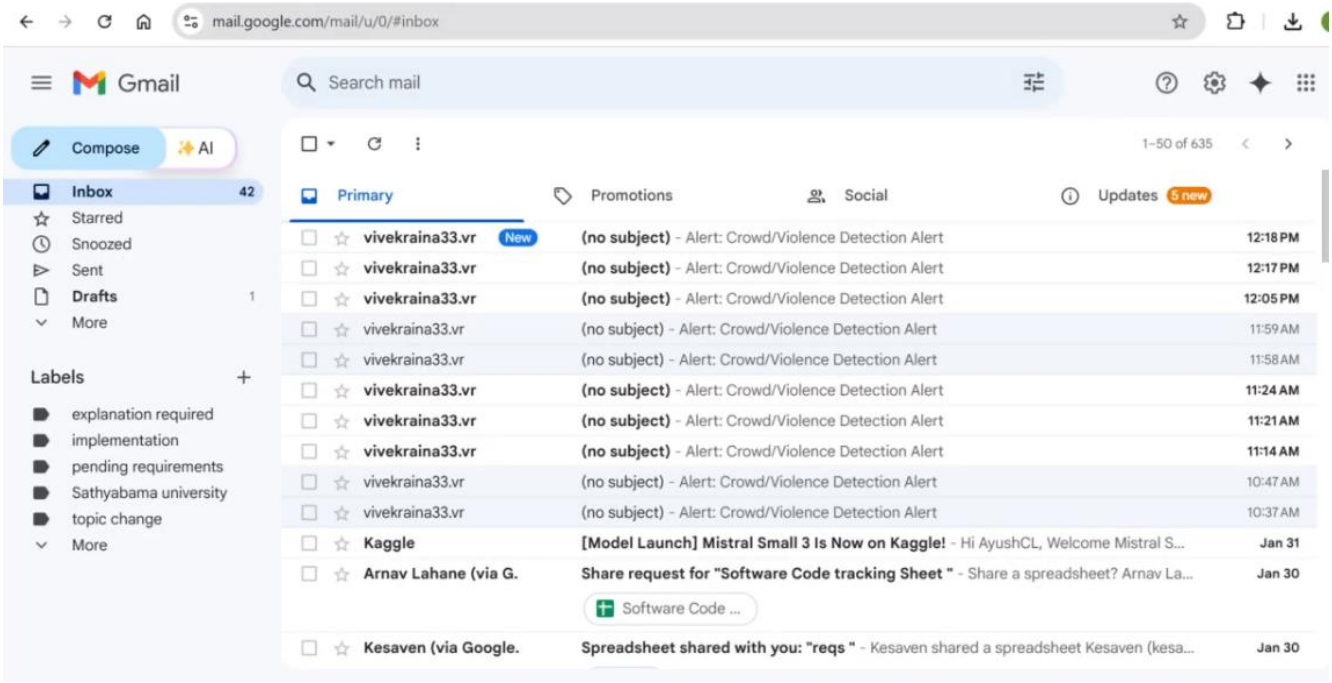
Crowd detection:



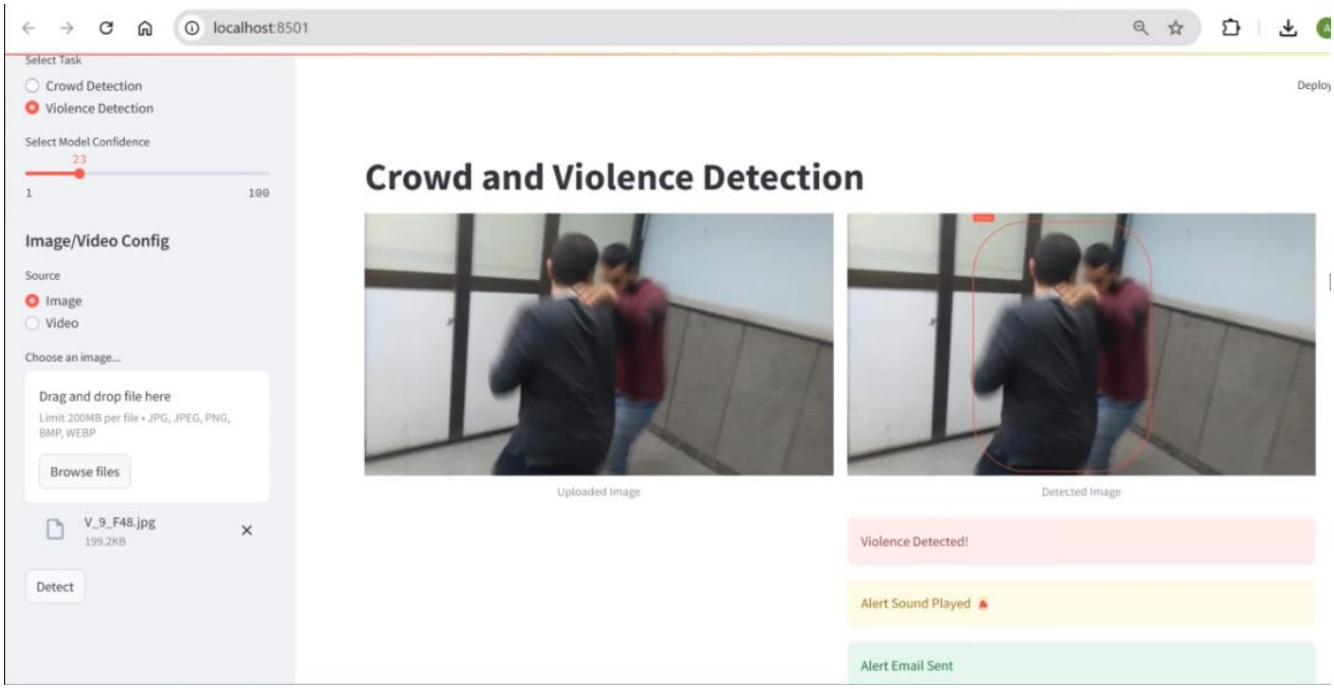


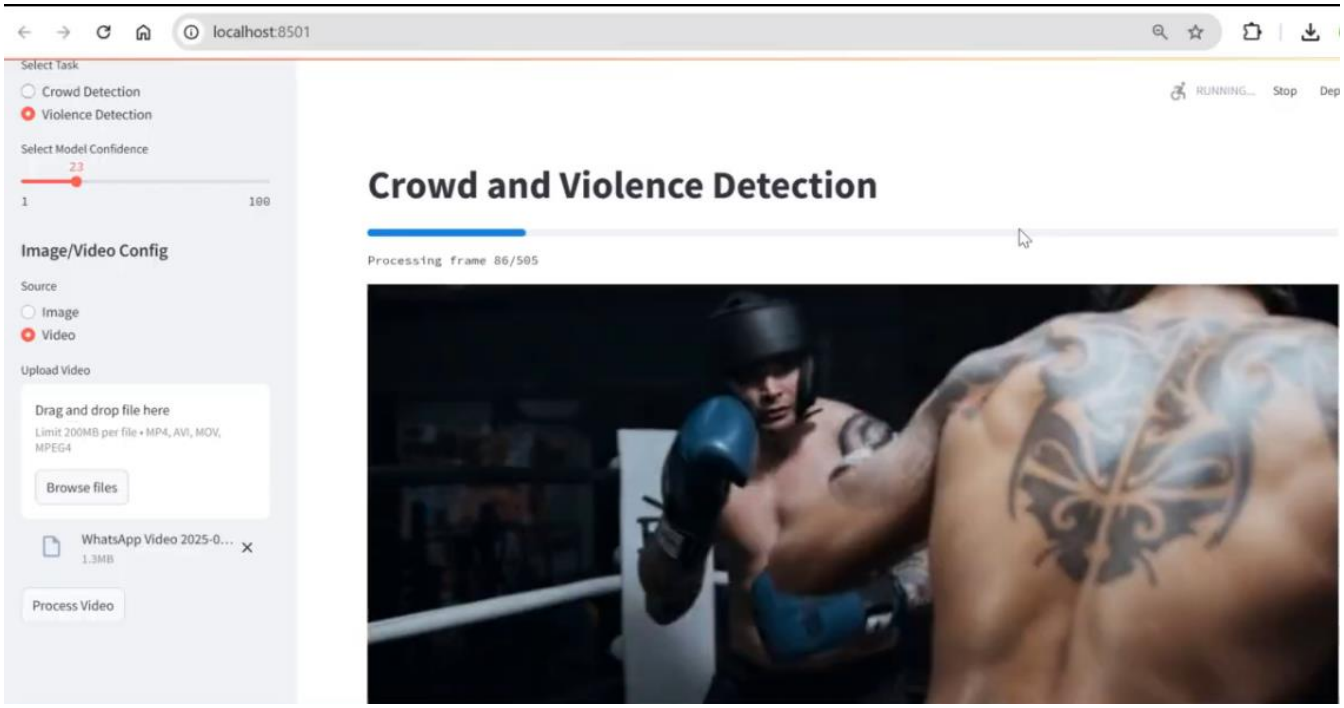


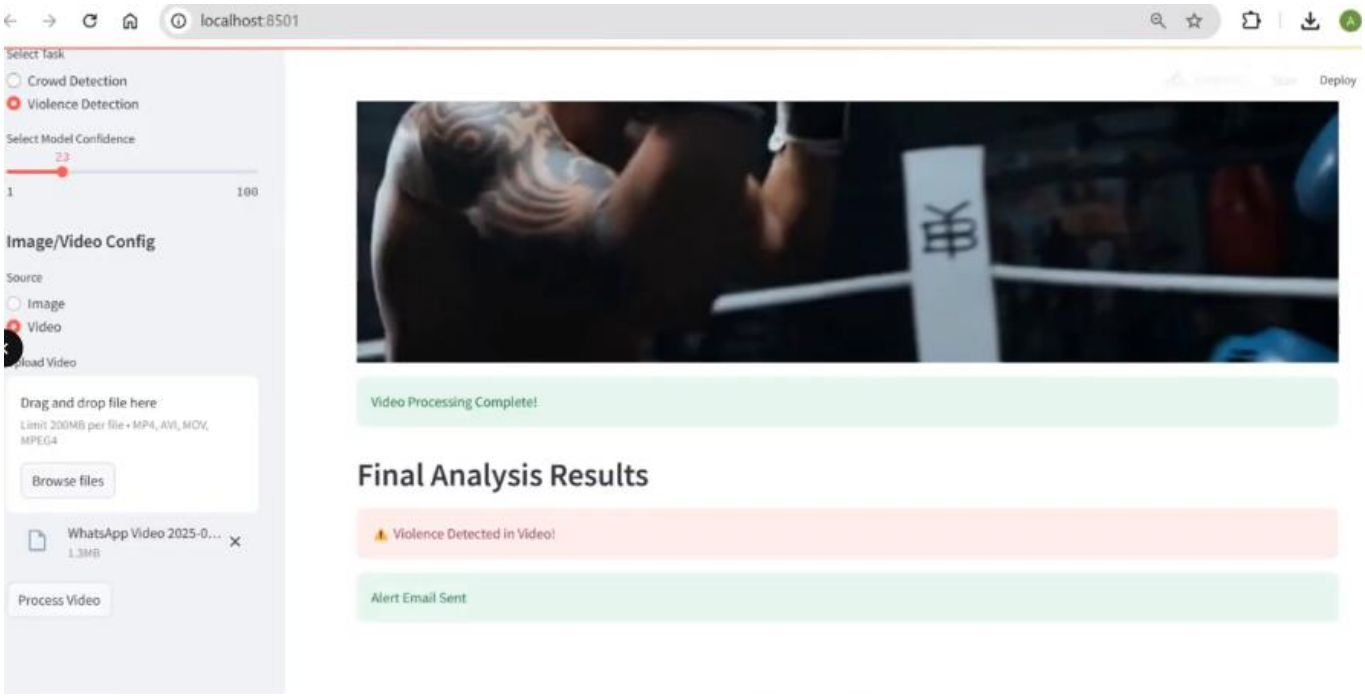
Alert through mail:



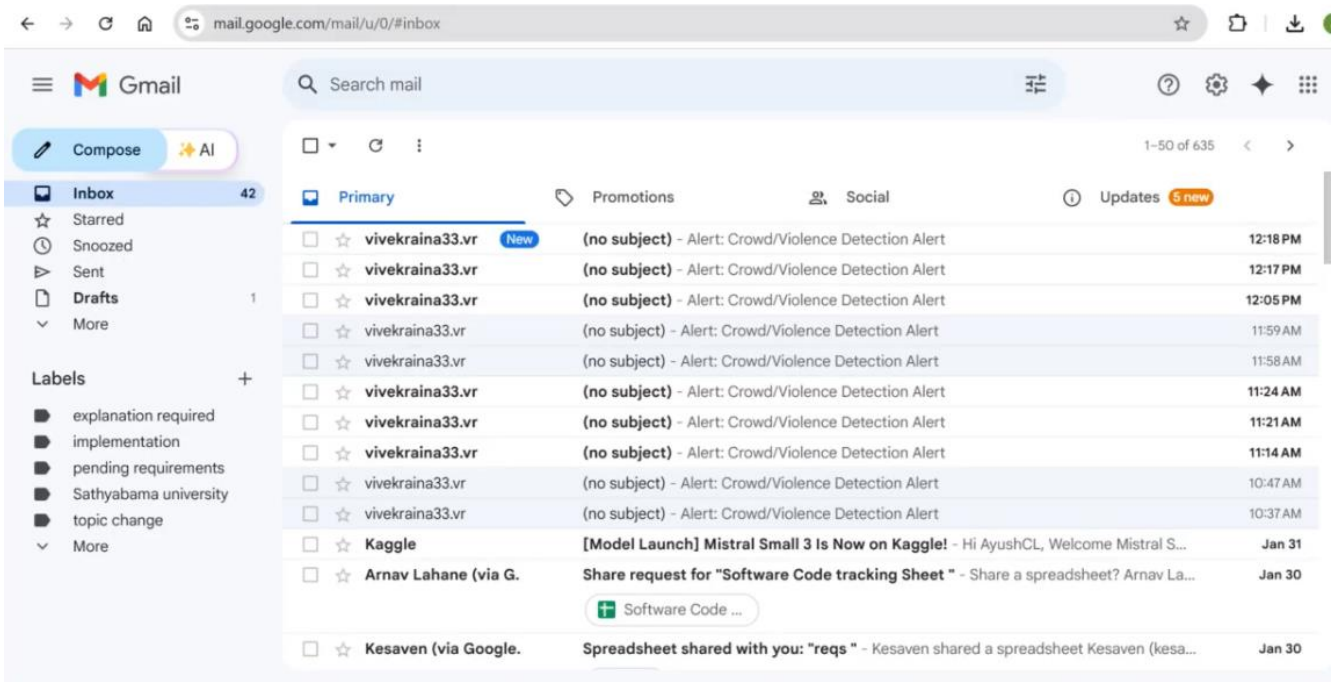
Violence detection:







Alert through mail:



APPENDIX 4: PAPER PLAGIARISM REPORT

REAL-TIME CROWD AND VIOLENCE DETECTION USING YOLOV8 ALGORITHM

Dr. KRISHNAMOORTHY M
Associate Professor
Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
hightechkrish2006@gmail.com

LOGESH S
Student
Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
lokeshsankar2003@gmail.com

LATHIKESHWARAN S
Student
Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
lathikesh7708214604@gmail.com

NAVEEN S
Student
Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
jpnaveen07@gmail.com

ABSTRACT

The proposed system aims to develop a real-time crowd and violence detection system using the state-of-the-art YOLOv8 algorithm. The system monitors live video feeds to detect sudden increases in crowd density and violent activities, providing real-time alerts for quick intervention. By leveraging the YOLOv8 model's advanced object detection capabilities, the system ensures high accuracy and speed in identifying potential security threats in public areas such as gatherings, events, and surveillance zones. This technology improves public safety by allowing authorities to respond quickly and proactively to incidents. The proposed system works by first collecting and preprocessing data from various sources, followed by training the YOLOv8 model on a curated dataset of diverse crowd and violence scenarios. This model is then integrated with a user-friendly interface that allows security personnel to interact with the system, receive notifications, and take action. Upon detecting an anomaly, the system triggers both an alert sound and an email notification, ensuring immediate attention. The system's ability to track crowd behavior and identify violent actions provides an enhanced security monitoring solution that is scalable and reliable. By combining real-time object detection, crowd density analysis, and violence detection into one integrated system, this project contributes significantly to the field of public safety and surveillance. This approach ensures that the system can handle diverse public safety challenges in various real-world settings.

I. INTRODUCTION

Public safety is a crucial concern in modern societies, especially in crowded public spaces such as transportation hubs, stadiums, and marketplaces. The rapid growth of urban populations has led to an increase in public gatherings, necessitating more efficient surveillance systems. Traditional security measures, including human monitoring and static CCTV surveillance, have limitations such as fatigue, errors, and delays in response time.with the advancement of artificial intelligence (AI) and computer vision, surveillance systems have evolved to incorporate automated monitoring. These systems help analyze human movement patterns, detect anomalies, and identify threats more accurately than manual methods. Machine learning models, especially deep learning algorithms, have significantly improved the efficiency of such systems by enabling automated, real-time threat detection.violence detection, in particular, has become a critical aspect of surveillance due to increasing incidents of criminal activities and public disturbances. Traditional surveillance cameras record events, but real-time response is lacking unless a human observer actively watches the footage. Delayed responses to violence and sudden crowd surges can lead to casualties, property damage, and chaos. To address these limitations, AI-driven solutions such as YOLO (You Only Look Once) have been employed for object detection, crowd analysis, and violence recognition. The latest iteration, YOLOv8, provides real-time detection capabilities with improved accuracy and efficiency. This project aims to

integrate YOLOv8 into a real-time monitoring system for public safety applications. By leveraging deep learning and real-time alert mechanisms, this project enhances security and response times in high-risk environments. The integration of automated email notifications and audio alerts ensures that security personnel and authorities can react promptly to potential threats.

II. LITERATURE SURVEY

[1] **Inbavalli A., Jarshini T., and Muralikrishnaa M.** proposed a concept in which an aggressive behavior detection and alert system was created using deep learning techniques. The system processes video feeds in real time to identify instances of aggressive behavior and trigger alerts. It employs Convolutional Neural Networks (CNNs) and other deep learning algorithms to ensure high accuracy and efficiency. This system enables immediate responses to violence or aggression in public spaces.

"EFFICIENT AGGRESSIVE BEHAVIOUR DETECTION AND ALERT SYSTEM EMPLOYING DEEP LEARNING TECHNIQUES – 2024"

Advantages - The system provides quick response times and high accuracy, helping to prevent violence escalation. By detecting aggressive behavior in real time, it enhances security and safety in public areas.

Disadvantages - The system's performance depends on the quality of the video feed. It may face challenges in highly crowded environments, where detecting individual aggressive actions becomes more difficult

[2] **Sudharson D., Srinithi J., Akshara S., Abhirami K., Sriharshitha P., and Priyanka K.** proposed a concept in which a proactive headcount and suspicious activity detection system was developed using the YOLOv8 algorithm. The system processes live video feeds to estimate crowd sizes and detect unusual behavior in public spaces. By identifying potential security threats in real time, it enables proactive safety measures.

"PROACTIVE HEADCOUNT AND SUSPICIOUS ACTIVITY DETECTION USING YOLOv8 – 2023"

Advantages - The system is highly effective for crowd control and monitoring. It can identify a variety of suspicious activities with high precision, improving public safety.

Disadvantages - The system may face difficulties in highly congested areas, where distinguishing between normal and suspicious behavior becomes challenging.

[3] **Nasir R., Jalil Z., Nasir M., Noor U., Ashraf M., and Saleem S.** proposed a concept in which an enhanced framework for real-time dense crowd abnormal behavior detection was developed using the YOLOv8 algorithm. The system is designed for dense crowd surveillance, identifying abnormal behaviors such as fights or panic in highly packed areas and triggering alerts for timely intervention. It employs data augmentation and advanced tracking techniques to enhance detection accuracy.

"AN ENHANCED FRAMEWORK FOR REAL-TIME DENSE CROWD ABNORMAL BEHAVIOR DETECTION USING YOLOv8 – 2024"

Advantages - The system is highly effective in crowded environments and significantly reduces false positives, making it reliable for dense crowd monitoring.

Disadvantages - The framework is computationally intensive, requiring powerful hardware for real-time processing and analysis.

[4] **Jyothsna V., Alle C., Kurnutala R., Ganesh K. N., KushalKarthik K. R., and Pydala B.** proposed a concept in which a YOLOv8-based security system was developed to detect individuals, monitor distances, and identify weapons or violent actions in real time. The system enhances security by providing speech alerts and email notifications to security personnel. By integrating multi-modal data, it improves detection accuracy and response efficiency.

"YOLOv8-BASED PERSON DETECTION, DISTANCE MONITORING, SPEECH ALERTS, AND WEAPON IDENTIFICATION WITH EMAIL NOTIFICATIONS – 2024"

Advantages - The system offers a robust, multi-layered security solution with real-time alerts and notifications, improving situational awareness and response time.

Disadvantages - The complexity of system integration requires significant resources for implementation and ongoing maintenance.

[5] **P., Patil S., and Pattanshetti T.** proposed a concept in which a real-time violence and weapon detection system was developed using advanced deep learning techniques. The system analyzes video footage from CCTV cameras to identify violent actions or weapons and triggers immediate alerts to security personnel. This approach enhances public safety by ensuring rapid response to potential threats.

"REAL-TIME VIOLENCE AND WEAPON DETECTION AND ALERT SYSTEM – 2024"

Advantages - The system achieves high accuracy in detecting threats and significantly reduces response time to incidents, improving public security.

Disadvantages - It may produce false positives in densely populated areas and could miss threats in low-quality video footage.

[6] **Kumar R., Rajpurohit D. S., Hanief M., Sharma S., Choudhury T., and Sar A.** proposed a concept in which a system was developed to detect criminal activities by analyzing live CCTV footage. The system leverages machine learning algorithms to identify specific criminal activities such as theft, assault, or vandalism and instantly notifies authorities for prompt intervention.

"DETECTION OF CRIMINAL ACTIVITIES/CRIMINAL THROUGH CCTV BY LIVE FOOTAGE ANALYSIS – 2024"

Advantages - The system enables early identification of criminal activities, significantly reducing response time and improving security measures.

Disadvantages - Its accuracy depends on the quality of CCTV footage and environmental conditions, which may affect detection efficiency.

[7] **Bensakhria A.** proposed a concept in which real-time edge AI video analytics was utilized for threat detection in sensitive environments such as airports, government buildings, and military sites. By processing video data at the edge, the system minimizes latency and enhances real-time decision-making, ensuring quicker responses to potential threats.

"LEVERAGING REAL-TIME EDGE AI-VIDEO ANALYTICS TO DETECT AND PREVENT THREATS IN SENSITIVE ENVIRONMENTS – 2023"

Advantages - The system enables real-time processing at the edge, reducing response time and improving overall security efficiency.

Disadvantages - The implementation requires high computational resources and comes with significant costs for deploying and maintaining edge devices.

[8] **Benoit P., Bresson M., Xing Y., Guo W., and Tsourdos A.** proposed a concept in which a real-time vision-based system was developed to detect violent actions through CCTV cameras using pose estimation. By analyzing human body movements, the system identifies aggression or violent behaviors in public spaces, enhancing public safety through automated surveillance.

"REAL-TIME VISION-BASED VIOLENT ACTIONS DETECTION THROUGH CCTV CAMERAS WITH POSE ESTIMATION – 2023"

Advantages - The system provides high precision in detecting violent actions by analyzing body pose and movement patterns.

Disadvantages - It may struggle with detecting actions when bodies are occluded or in low-light conditions, affecting accuracy.

[9] Reddy K. and Reddy S. proposed a concept in which a smart monitoring system, OccupEye, was developed to track building traffic and animal movement using AI-based video analysis. The system identifies patterns in both human and animal movements, making it useful for wildlife monitoring and efficient building occupancy management.

"OCCUPEYE – BUILDING TRAFFIC AND ANIMAL MONITORING SYSTEM – 2024"

Advantages - The system is versatile, serving dual purposes in wildlife tracking and building management, enhancing efficiency in both domains.

Disadvantages - Implementing the system in complex environments with varied operational conditions may pose challenges to accuracy and reliability.

[10] **Sapkota R., Qureshi R., Calero M. F., Badjugar C., Nepal U., Poulose A., and Karkee M.** proposed a comprehensive review detailing the evolution of the YOLO object detection algorithm from its inception to the latest YOLOv10 version. The study highlights advancements in accuracy, speed, and diverse applications in real-time object detection tasks, providing a thorough analysis of the algorithm's development.

"YOLOv10 TO ITS GENESIS: A DECADAL AND COMPREHENSIVE REVIEW OF THE YOU ONLY LOOK ONCE (YOLO) SERIES – 2024"

Advantages - Offers an in-depth understanding of the YOLO algorithm, its improvements, and its capabilities across different versions.

Disadvantages - The review focuses on theoretical advancements and is not directly applicable to specific real-world problems without further adaptation.

[11] **Qaraqe M., Yang Y. D., Varghese E. B., Basaran E., and Elzein A.** proposed a concept in which the Swin Transformer was leveraged to analyze crowd size and detect violent behaviors in real-time. The system processes video data to assess both the density and potential threat level of a crowd, improving situational awareness and security management.

"CROWD BEHAVIOR DETECTION: LEVERAGING VIDEO SWIN TRANSFORMER FOR CROWD SIZE AND VIOLENCE LEVEL ANALYSIS – 2024"

Advantages - The use of the Swin Transformer enhances performance in both crowd detection and violence level analysis, ensuring higher accuracy in real-world scenarios.

Disadvantages - The system demands significant computational resources for real-time processing, making deployment challenging in resource-constrained environments.

Fig no 1: Data Flow Diagram

III. EXISTING SYSTEM:

Traditional surveillance systems rely on human monitoring, whiere security personnel manually observe live or recorded video feeds to detect suspicious activities. While this method has been in use for decades, it suffers from sevel limitaions, such as human fatigue, observational biases, and delayed response times. As a result, security breaches and violent incidents often go unnoticed until it is too late to intervence effectively. Many security systems use basic motion detection algorithms that analyze pixel changes in consecutive frames. However, these methods are highly prone to false alarms, especially in crowded places where natural movements of people can trigger alerts. Additionally, they lack the ability to differentiate between normal crowd behavior and violent activities, making them unreliable for public safety monitoring. Another major drawback of conventional surveillance systems is their lack of real-time responsiveness. Most security footage is reviewed after an incident has occurred, which limits the ability to prevent crimes or respond quickly to violent situations. This results in significant delays in law enforcement actions, increasing the risk to public safety. Finally, lack of integration with alert mechanisms in current systems reduces their effectiveness. Even if an AI model detects a potential threat, many systems do not provide immediate notifications to security personnel. This means that without an operator actively watching the feed, threats may go unnoticed for extended periods, leading to security failures. Traditional surveillance systems face multiple challenges that hinder their effectiveness in ensuring security and rapid incident response.

Manual monitoring ineffeciencies: Surveillance often requires continuous human oversight, relying on security personnel to manually analyze footage for potential threats. This approach is costly, labor-intensive, and highly prone to fatigue, which can compromise vigilance and response times. Inconsistencies in threat assessment due to human interpretation further increase the risk of critical incidents being overlooked. Limited Real-Time Processing: While some systems integrate motion detection and behavioral analysis, they struggle with real-time data processing. The high volume and velocity of video streams often overwhelm these systems, leading to significant delays between incident occurrence and detection. Such delays can be critical in emergency situations where rapid intervention is essential. Inaccuracy and False Alarms: Many conventional surveillance systems rely on outdated algorithms that fail to distinguish normal activities from genuine threats accurately. This results in frequent false positives—triggering unnecessary alarms—and false negatives—failing to detect actual dangers. Over time, these inaccuracies erode trust in the system, desensitize operators, and create inefficiencies in security management. Lack of Integrated Responses: Most traditional surveillance systems operate in isolation, capable of generating alerts but lacking automated integration with emergency response mechanisms. This means security teams must manually verify and escalate incidents, delaying response times and increasing the potential for lapses in crisis management.

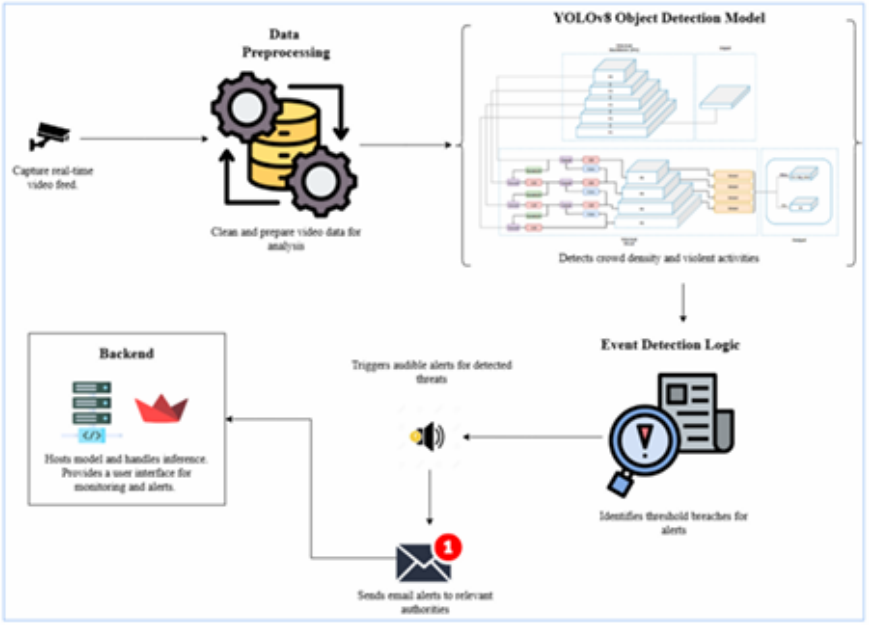


Fig no 2: Architecture Diagram

IV. PROPOSED SYSTEM

The proposed surveillance system integrates the advanced capabilities of YOLOv8, enabling real-time detection of crowd density fluctuations and violent activities to enhance security in public settings. As the latest iteration in the You Only Look Once algorithm series, YOLOv8 leverages cutting-edge deep learning techniques to process high-resolution video feeds in real time, ensuring immediate identification of potential threats and abnormal behaviors. A key advancement of this system is its real-time anomaly and violence detection, powered by deep convolutional neural networks that process and analyze video frames at high speeds without compromising accuracy. Unlike traditional models, YOLOv8 can distinguish subtle nuances in crowd movements and detect abrupt behavioral changes, providing precise alerts almost instantaneously. This capability is crucial in preventing or mitigating security incidents, allowing immediate action before situations escalate. Additionally, the system incorporates an automated alert mechanism that activates audible alarms within the monitored environment and dispatches email notifications to designated authorities or security personnel. This dual-alert feature ensures both onsite and remote teams are informed in real time, facilitating a swift, coordinated response while reducing dependency on constant human oversight. Security staff can focus on strategic intervention rather than continuous monitoring, improving operational efficiency. The benefits of the system are substantial, starting with increased accuracy and reduced false alarms. The advanced deep learning models within YOLOv8 significantly enhance precision, minimizing false positives and negatives, thus improving reliability and conserving security resources. By reducing unnecessary alerts, security personnel can focus on genuine threats, ensuring efficient surveillance protocols. Moreover, the system's rapid detection and alert capabilities play a vital role in enhancing public safety by enabling quick responses to potential dangers, preventing escalations, and effectively managing public spaces. The system can be tailored to specific requirements, including adjustments in camera configurations and processing power, ensuring optimal performance across different operational scales. This flexibility makes it a versatile solution for diverse security needs, reinforcing its effectiveness in dynamic public environments. By leveraging the power of YOLOv8, the proposed system revolutionizes security surveillance with real-time detection, automated alerts, an intuitive interface, continuous improvement, and unparalleled scalability, collectively strengthening public safety measures with cutting-edge technology.

V. METHODOLOGY

a) YOLO Object Detection Model

You Only Look Once (YOLO) is a deep learning-based object detection algorithm optimized for real-time applications. Unlike traditional methods like R-CNN, which scan images multiple times, YOLO processes the entire image in a single pass, significantly improving detection speed. The algorithm employs a grid-based approach, dividing the image into multiple cells. Each cell predicts bounding boxes, class probabilities, and confidence scores simultaneously. This enables the model to detect multiple objects in a single frame, reducing computational complexity. YOLOv8 introduces several enhancements for higher precision and recall, making it more efficient for tasks such as violence detection and crowd monitoring. Improvements include: Better feature extraction for detecting smaller objects, Optimized anchor box assignments for more accurate bounding box predictions and Enhanced neural network layers for faster and more efficient processing.

Fig no 3: Data Preprocessing Steps for Image Processing

b) Data Preprocessing in Deep Learning

Data preprocessing is a crucial step in training deep learning models, as raw data often contains noise, inconsistencies, and irrelevant information. Preprocessing ensures that the data is clean, structured, and optimized for better learning performance.For image-based models like YOLOv8, preprocessing involves image resizing, normalization, augmentation, and feature extraction. These steps help in improving model generalization and accuracy.

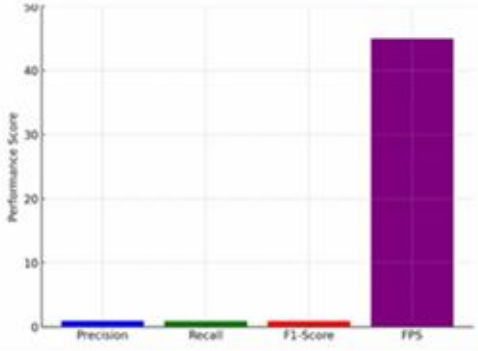


Fig no 4: YoloV8 Object Detection Performance Metrics

c) Data Augmentation Techniques

Data augmentation involves artificially expanding the training dataset by applying transformations such as rotation, flipping, brightness adjustments, and noise

addition. This helps the model learn robust features, improving its performance on real-world data. For this project, images in the dataset are augmented with different lighting conditions, angles, and resolutions, allowing the model to detect violent activities in various environments.

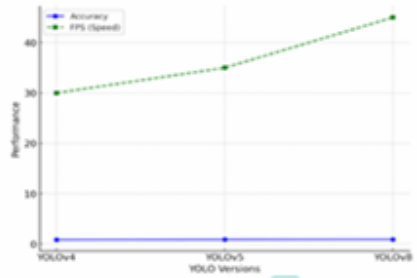


Fig no 5: Comparison of YOLO Versions in Terms of Accuracy and Speed

d) Feature Extraction and Object Localization

Feature extraction involves identifying key patterns in an image, such as edges, textures, and object contours. Deep learning models use convolutional layers to extract these features automatically.For violence detection, features such as body posture, hand movements, and crowd density patterns are extracted. These features help differentiate normal behaviors from aggressive actions, improving detection accuracy.

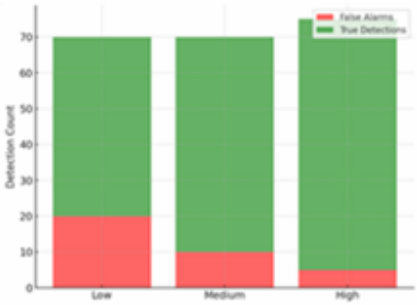


Fig no 6: Alert System Activation Rate based on Detection Thresholds

VI. DATABASE DESIGN

The database for the real-time crowd and violence detection system is designed for high-throughput and rapid data retrieval to efficiently handle video streams and analytics. It consists of several key tables: The Videos Table stores metadata about video feeds, including VideoID, CameraID, StartTime, EndTime, and StoragePath, enabling efficient indexing and retrieval. The Detections Table logs each event detected by the YOLOv8 algorithm, with fields like DetectionID, VideoID (foreign key), Timestamp, Type (e.g., crowd increase, violent act), and ConfidenceScore, supporting real-time alerts and historical analysis. The Alerts Table tracks issued alerts, containing AlertID, DetectionID (foreign key), SentTime, and ResponseStatus, ensuring accountability. The Users Table manages system operators with fields such as UserID, Username, PasswordHash, Email, and Role (e.g., administrator, viewer), supporting access control. Lastly, the Logs Table records system operations and user activities, storing LogID, UserID (foreign key), ActivityType, Description, and Timestamp, aiding troubleshooting and monitoring.

VII. CONCLUSION AND FUTURE WORK

Future enhancements will focus on integrating audio-based violence detection to identify gunshots, screams, and distress signals, improving accuracy through multimodal analysis. Additionally, edge computing and IoT-enabled real-time processing will allow deployment on low-power devices, making the system more scalable for smart city surveillance networks. Further improvements in predictive analytics, AI-driven behavioral analysis, and multi-camera synchronization will enhance threat anticipation and coordinated security responses.

Cloud-based deployment on platforms like AWS and Google Cloud will enable remote access for law enforcement agencies, ensuring faster decision-making and incident response. These advancements will make the system more flexible and accessible across different security infrastructures, allowing seamless integration with existing surveillance frameworks. With improved AI models, real-time processing, and cloud support, the system aims to enhance both security efficiency and incident management.

The real-time crowd and violence detection system using YOLOv8 represents a major advancement in AI-powered surveillance, offering automated real-time detection and alert mechanisms to enhance public safety and security operations. By leveraging deep learning and computer vision, the system efficiently identifies sudden crowd density changes and violent activities, ensuring rapid response by security personnel. The integration of real-time notifications, sound alarms, and an interactive monitoring dashboard minimizes the reliance on manual surveillance, reducing response time and improving situational awareness. With high accuracy and processing speed, the system is adaptable for deployment in high-risk areas such as public events, transportation hubs, and commercial spaces. However, challenges such as false positives in densely populated areas, sensitivity to varying lighting conditions, and computational resource requirements highlight the need for further optimization.

VIII. REFERENCE

[1]. A. Inbavalli, T. Jarshini and M. Muralikrishnaa, "Efficient Aggressive Behaviour Detection And Alert System Employing Deep Learning Techniques," in 2024 International Conference on Automation and Computation (AUTOCOM), pp. 404-410, March 2024, IEEE.

[2]. D. Sudharson, J. Srinithi, S. Akshara, K. Abhirami, P. Sriharshitha and K. Priyanka, "Proactive Headcount and

Suspicious Activity Detection using YOLOv8," in Procedia Computer Science, vol. 230, pp. 61-69, 2023.

[3]. R. Nasir, Z. Jalil, M. Nasir, U. Noor, M. Ashraf, and S. Saleem, "An Enhanced Framework for Real-Time Dense Crowd Abnormal Behavior Detection Using YOLOv8," 2024.

[4]. V. Jyothsna, C. Alle, R. Kurnutala, K. N. Ganesh, K. R. KushalKarthik, and B. Pydala, "YOLOv8-Based Person Detection, Distance Monitoring, Speech Alerts, and Weapon Identification with Email Notifications," in 2024 International Conference on Expert Clouds and Applications (ICOECA), pp. 288-296, April 2024, IEEE.

[5]. P. Patil, S. Patil, and T. Pattanshetti, "Real-Time Violence and Weapon Detection and Alert System – 2024." This system utilizes advanced deep learning techniques to analyze CCTV footage, detecting violent actions or weapons and triggering immediate alerts for enhanced public safety.

[6]. R. Kumar, D. S. Rajpurohit, M. Hanief, S. Sharma, T. Choudhury, and A. Sar, "Detection Of Criminal Activities/Criminal Through CCTV By Live Footage Analysis," in 2024 OPJU International Technology Conference (OTCON) on Smart Computing for Innovation and Advancement in Industry 4.0, pp. 1-6, June 2024, IEEE.

[7]. A. Bensakhria, "Leveraging Real-Time Edge AI-Video Analytics to Detect and Prevent Threats in Sensitive Environments," in 2023 [Conference Name], pp. 1-6, IEEE.

[8]. P. Benoit, M. Bresson, Y. Xing, W. Guo, and A. Tsourdos, "Real-Time Vision-Based Violent Actions Detection Through CCTV Cameras With Pose Estimation," in 2023 IEEE Smart World Congress (SWC), pp. 844-849, August 2023, IEEE.

[9]. K. Reddy and S. Reddy, "OccupEye-Building Traffic and Animal Monitoring System," in Proceedings of the EAI 3rd International Conference on Intelligent Systems and Machine Learning (ICISML 2024), January 5-6, 2024, Pune, India, August 2024.

[10]. R. Sapkota, R. Qureshi, M. F. Calero, C. Badjugar, U. Nepal, A. Poulose, et al., "YOLOv10 to Its Genesis: A Decadal and Comprehensive Review of The You Only Look Once (YOLO) Series," arXiv preprint, arXiv:2406.19407, 2024.

[11]. M. Qaraqe, Y. D. Yang, E. B. Varghese, E. Basaran, and A. Elzein, "Crowd behavior detection: leveraging video swin transformer for crowd size and violence level analysis," Applied Intelligence, vol. 54, no. 21, pp. 10709-10730, 2024.

[12]. K. Gkountakos, K. Ioannidis, T. Tsikrika, S. Vrochidis, and I. Kompatsiaris, "Crowd Violence Detection from Video Footage," in 2021 International Conference on Content-Based Multimedia Indexing (CBMI), Lille, France, pp. 1-4, 2021, doi: 10.1109/CBMI50038.2021.9461921.

[13]. K. C. Chang and Y.-C. Liao, "Design of Violence Event Detection System Based on CCTVs by Human Body Pose Recognition," in 2023 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan), PingTung, Taiwan, pp. 695-696, 2023, doi: 10.1109/ICCE-Taiwan58799.2023.10226669.

[14]. A. S. Hada, B. J. S. Bainsla, S. Tapaswi, and B. Jana, "Embedded System for Societal Violence Detection and Intervention," in 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, pp. 1-6, 2024, doi: 10.1109/CONECCT62155.2024.10677210.

[15]. A. O. Salau, N. I. Daniel, O. Ayobamidele, S. K. Vohra, A. Alemran, and M. O. Onibonoje, "Enhancing Hostel Security: Integration of Deep Learning Models for Smoking and Violence Detection," in 2024 Second International Conference on Computational and Characterization Techniques in Engineering & Sciences (IC3TES), Lucknow, India,pp.17,2024,doi:10.11.09/IC3TES62412.2024.1087571.

ORIGINALITY REPORT

8%	7%	9%	8%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.ijecs.in Internet Source	1%
2	V. Sharmila, S. Kannadhasan, A. Rajiv Kannan, P. Sivakumar, V. Vennila. "Challenges in Information, Communication and Computing Technology", CRC Press, 2024 Publication	1%
3	thesai.org Internet Source	1%
4	Ntandoyenkosi Zungu, Peter Olukanmi, Pitshou Bokoro. "SynthSecureNet: An Improved Deep Learning Architecture with Application to Intelligent Violence Detection", Algorithms, 2025 Publication	1%
5	researchportal.hbku.edu.qa Internet Source	1%
6	amity.edu Internet Source	
7	G. Sucharitha, Anjanna Matta, M. Srinivas, Sachi Nandan Mohanty. "Artificial Intelligence Technologies for Engineering Applications", CRC Press, 2025 Publication	1%
8	Submitted to University of Auckland Student Paper	1%
9	Submitted to Noakhali Science and Technology University Student Paper	1%

10	repositorio.uloyola.es Internet Source	1 %
11	Submitted to Canterbury Christ Church University Student Paper	<1 %
12	Submitted to Loyola University, Chicago Student Paper	<1 %
13	kct.ac.in Internet Source	<1 %
14	Submitted to Liverpool John Moores University Student Paper	<1 %
15	Submitted to University of Teesside Student Paper	<1 %
16	Mariam Nasim, Rafia Mumtaz, Muneer Ahmad, Arshad Ali. "Fabric Defect Detection in Real World Manufacturing Using Deep Learning", Information, 2024 Publication	<1 %
17	dspace.lib.cranfield.ac.uk Internet Source	<1 %
18	"Contents", Procedia Computer Science, 2023 Publication	<1 %
19	Submitted to University of Wales Institute, Cardiff Student Paper	<1 %
20	Volodymyr Tyvoniuk, Roman Trach, Yuliia Trach. "Integration of Probability Maps into Machine Learning Models for Enhanced Crack Segmentation in Concrete Bridges", Applied Sciences, 2025 Publication	<1 %
21	www.cranfield.ac.uk Internet Source	<1 %

22	Rahima Khanam, Tahreem Asghar, Muhammad Hussain. "Comparative Performance Evaluation of YOLOv5, YOLOv8, and YOLOv11 for Solar Panel Defect Detection", Solar, 2025 Publication	<1 %
23	ebin.pub Internet Source	<1 %
24	R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P. Prasad. "Algorithms in Advanced Artificial Intelligence - Proceedings of International Conference on Algorithms in Advanced Artificial Intelligence (ICAAAI-2024)", CRC Press, 2025 Publication	<1 %
25	ijnrd.org Internet Source	<1 %
26	Pablo Guarnido-Lopez, John-Fredy Ramirez-Agudelo, Emmanuel Denimal, Mohammed Benaouda. "Programming and Setting Up the Object Detection Algorithm YOLO to Determine Feeding Activities of Beef Cattle: A Comparison between YOLOv8m and YOLOv10m", Animals, 2024 Publication	<1 %
27	Sudharson D, Srinithi J, Akshara S, Abhirami K, Sriharshitha P, Priyanka K. "Proactive Headcount and Suspicious Activity Detection using YOLOv8", Procedia Computer Science, 2023 Publication	<1 %

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		

REFERENCE :

[1] Rajini S, Dhivyaa A S, Jananipriya R M, Karthika priya V S, “VOICE-BASED EMAIL SYSTEM FOR VISUALLY IMPAIRED PEOPLE”.International Research Journal of Engineering and Technology (IRJET), Volume: 09, Issue: 06, 2022.

[2]. D. Sudharson, J. Srinithi, S. Akshara, K. Abhirami, P. Sriharshitha and K. Priyanka, "Proactive Headcount and Suspicious Activity Detection using YOLOv8," in Procedia Computer Science, vol. 230, pp. 61-69, 2023.

[3]. R. Nasir, Z. Jalil, M. Nasir, U. Noor, M. Ashraf, and S. Saleem, "An Enhanced Framework for Real-Time Dense Crowd Abnormal Behavior Detection Using YOLOv8," 2024.

[4]. V. Jyothsna, C. Alle, R. Kurnutala, K. N. Ganesh, K. R. KushalKarthik, and B. Pydala, "YOLOv8-Based Person Detection, Distance Monitoring, Speech Alerts, and Weapon Identification with Email Notifications," in 2024 International Conference on Expert Clouds and Applications (ICOECA), pp. 288-296, April 2024, IEEE.

[5]. P. Patil, S. Patil, and T. Pattanshetti, "Real-Time Violence and Weapon Detection and Alert System – 2024." This system utilizes advanced deep learning techniques to analyze CCTV footage, detecting violent actions or weapons and triggering immediate alerts for enhanced public safety.

[6]. R. Kumar, D. S. Rajpurohit, M. Hanief, S. Sharma, T. Choudhury, and A. Sar, "Detection Of Criminal Activities/Criminal Through CCTV By Live Footage Analysis," in 2024 OPJU International Technology Conference (OTCON) on Smart Computing for Innovation and Advancement in Industry 4.0, pp. 1-6, June 2024, IEEE.

[7]. A. Bensakhria, "Leveraging Real-Time Edge AI-Video Analytics to Detect and Prevent Threats in Sensitive Environments," in 2023 [Conference Name], pp. 1-6, IEEE.

[8]. P. Benoit, M. Bresson, Y. Xing, W. Guo, and A. Tsourdos, "Real-Time Vision-Based Violent Actions Detection Through CCTV Cameras With Pose Estimation," in 2023 IEEE Smart World Congress (SWC), pp. 844-849, August 2023, IEEE

[9]. K. Reddy and S. Reddy, "OccupEye-Building Traffic and Animal Monitoring System," in Proceedings of the EAI 3rd International Conference on Intelligent Systems and Machine Learning (ICISML 2024), January 5-6, 2024, Pune, India, August 2024.

[10]. R. Sapkota, R. Qureshi, M. F. Calero, C. Badjugar, U. Nepal, A. Poulose, et al., "YOLOv10 to Its Genesis: A Decadal and Comprehensive Review of The You Only Look Once (YOLO) Series," arXiv preprint, arXiv:2406.19407, 2024.

[11]. M. Qaraqe, Y. D. Yang, E. B. Varghese, E. Basaran, and A. Elzein, "Crowd behavior detection: leveraging video swin transformer for crowd size and violence level analysis," Applied Intelligence, vol. 54, no. 21, pp. 10709-10730, 2024.

[12]. K. Gkountakos, K. Ioannidis, T. Tsikrika, S. Vrochidis, and I. Kompatsiaris, "Crowd Violence Detection from Video Footage," in 2021 International Conference on Content-Based Multimedia Indexing (CBMI), Lille, France, pp. 1-4, 2021, doi: 10.1109/CBMI50038.2021.9461921.

[13]. K. C. Chang and Y.-C. Liao, "Design of Violence Event Detection System Based on CCTVs by Human Body Pose Recognition," in 2023 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan), PingTung, Taiwan, pp. 695-696, 2023, doi: 10.1109/ICCE-Taiwan58799.2023.10226669.

[14]. A. S. Hada, B. J. S. Bainsla, S. Tapaswi, and B. Jana, "Embedded System for Societal Violence Detection and Intervention," in 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, pp. 1-6, 2024, doi: 10.1109/CONECCT62155.2024.10677210.

[15]. A. O. Salau, N. I. Daniel, O. Ayobamidele, S. K. Vohra, A. Alemran, and M. O. Onibonoje, "Enhancing Hostel Security: Integration of Deep Learning Models for Smoking and Violence Detection," in 2024 Second International Conference on Computational and Characterization Techniques in Engineering & Sciences (IC3TES), Lucknow, India,pp.17,2024,doi:10.11.09/IC3TES62412.2024.1087571.